

Digital Design & Computer Arch.

Lecture 24a: Advanced Caches

Prof. Onur Mutlu

ETH Zürich

Spring 2026

22 May 2026

Readings for This Week and Next Week

- Memory Hierarchy, Caches, Prefetching, Virtual Memory
- Required
 - H&H Chapters 8.1-8.3
 - Refresh: P&P Chapter 3.5
 - Kim & Mutlu, "**Memory Systems**," Computing Handbook, 2014.
 - https://people.inf.ethz.ch/omutlu/pub/memory-systems-introduction_computing-handbook14.pdf
- Recommended
 - An early cache paper by Maurice Wilkes
 - Wilkes, "**Slave Memories and Dynamic Storage Allocation**," IEEE Trans. On Electronic Computers, 1965.
 - An example prefetching paper
 - Mutlu et al., "**Runahead Execution: An Effective Alternative to Large Instruction Windows**," IEEE Micro, 2003.

Cache Performance

Recall: How to Improve Cache Performance

- Three fundamental goals
- Reducing miss rate
 - Caveat: reducing miss rate can reduce performance if more costly-to-refetch blocks are evicted
- Reducing miss latency or miss cost
- Reducing hit latency or hit cost
- The above three **together** affect performance

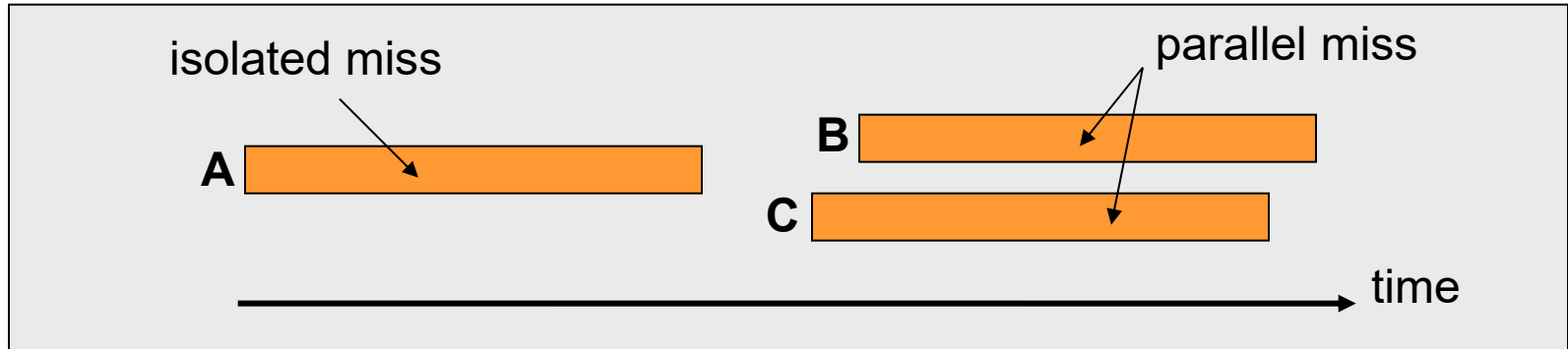
Improving Basic Cache Performance

- Reducing miss rate
 - ❑ More associativity
 - ❑ Alternatives/enhancements to associativity
 - Victim caches, hashing, pseudo-associativity, skewed associativity
 - ❑ Better replacement/insertion policies
 - ❑ **Software approaches**
- Reducing miss latency/cost
 - ❑ Multi-level caches
 - ❑ Critical word first
 - ❑ Subblocking/sectoring
 - ❑ **Better replacement/insertion policies**
 - ❑ Non-blocking caches (multiple cache misses in parallel)
 - ❑ Multiple accesses per cycle
 - ❑ **Software approaches**

Miss Latency/Cost

- What is miss latency or miss cost affected by?
 - Where does the miss get serviced from?
 - What level of cache in the hierarchy?
 - Row hit versus row conflict in DRAM (bank/rank/channel conflict)
 - Queueing delays in the memory controller and the interconnect
 - Local vs. remote memory (chip, node, rack, remote server, ...)
 - ...
 - How much does the miss stall the processor?
 - Is it overlapped with other latencies?
 - Is the data immediately needed by the processor?
 - Is the incoming block going to evict a longer-to-refetch block?
 - ...

Memory Level Parallelism (MLP)



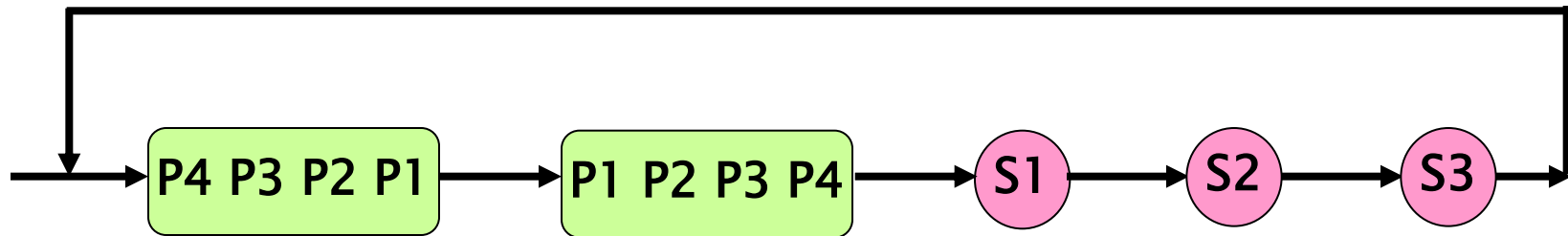
- ❑ Memory Level Parallelism (MLP) is the notion of generating and servicing multiple memory accesses in parallel [Glew'98]
- ❑ Several techniques improve MLP (e.g., out-of-order execution)
- ❑ MLP varies. Some misses are isolated and some parallel

How does this affect cache replacement?

Traditional Cache Replacement Policies

- ❑ Traditional cache management policies try to reduce miss count
- ❑ **Implicit assumption**: Reducing miss count reduces memory-related stall time
- ❑ Misses with varying cost/MLP **breaks** this assumption!
- ❑ Eliminating an isolated miss helps performance more than eliminating a parallel miss
- ❑ Eliminating a higher-latency miss could help performance more than eliminating a lower-latency miss

An Example



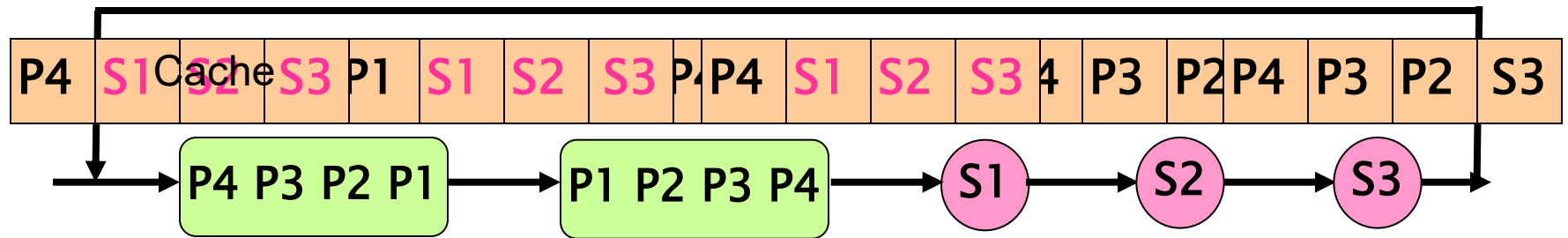
Misses to blocks P1, P2, P3, P4 can be serviced in parallel
Misses to blocks S1, S2, and S3 are isolated (i.e., serviced serially)

Two replacement algorithms:

1. Minimizes miss count (Belady's OPT)
2. Reduces isolated miss (MLP-Aware)

For a fully associative cache containing 4 blocks

Fewest Misses $\neq$ Best Performance



Hit/Miss H H H M H H H H M M M



Misses=4
Stalls=4

Belady's OPT replacement

Hit/Miss H M M M H M M M H H H



Saved
cycles

Misses=6
Stalls=2

MLP-Aware replacement

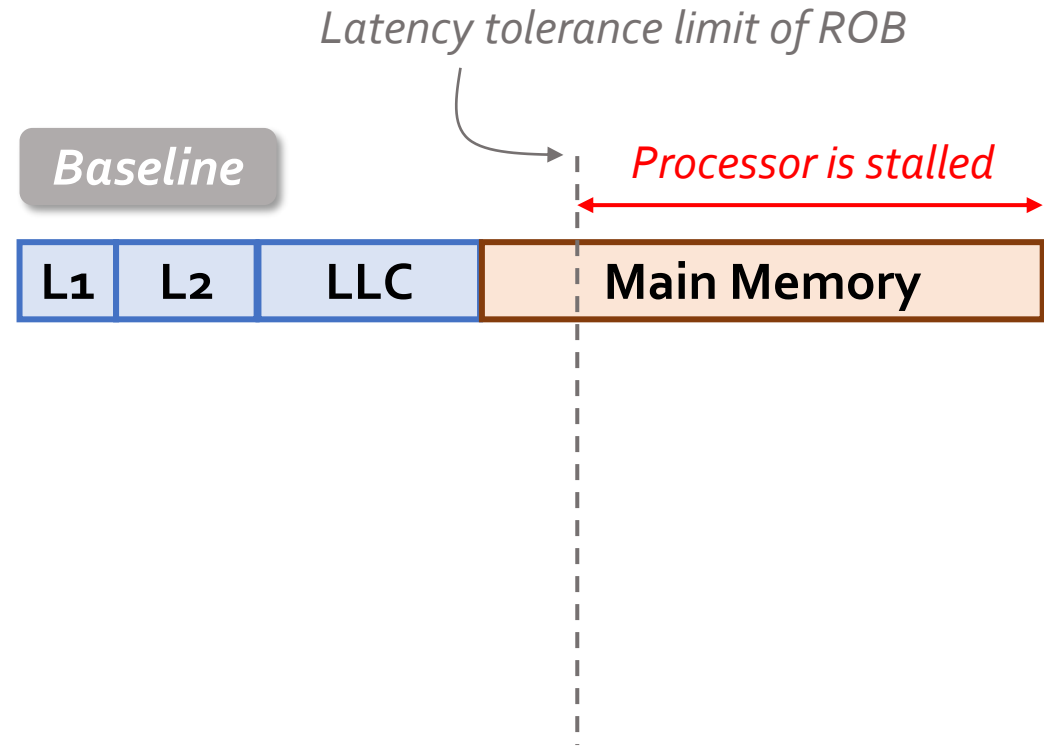
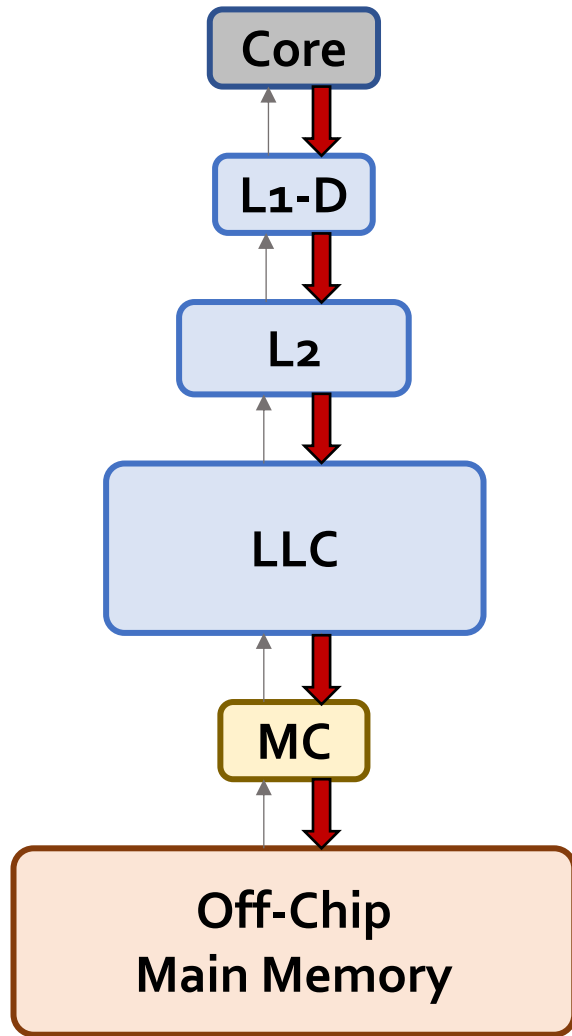
Recommended: MLP-Aware Cache Replacement

- How do we incorporate MLP/cost into replacement decisions?
- How do we design a hybrid cache replacement policy?
- Qureshi et al., "A Case for MLP-Aware Cache Replacement," ISCA 2006.

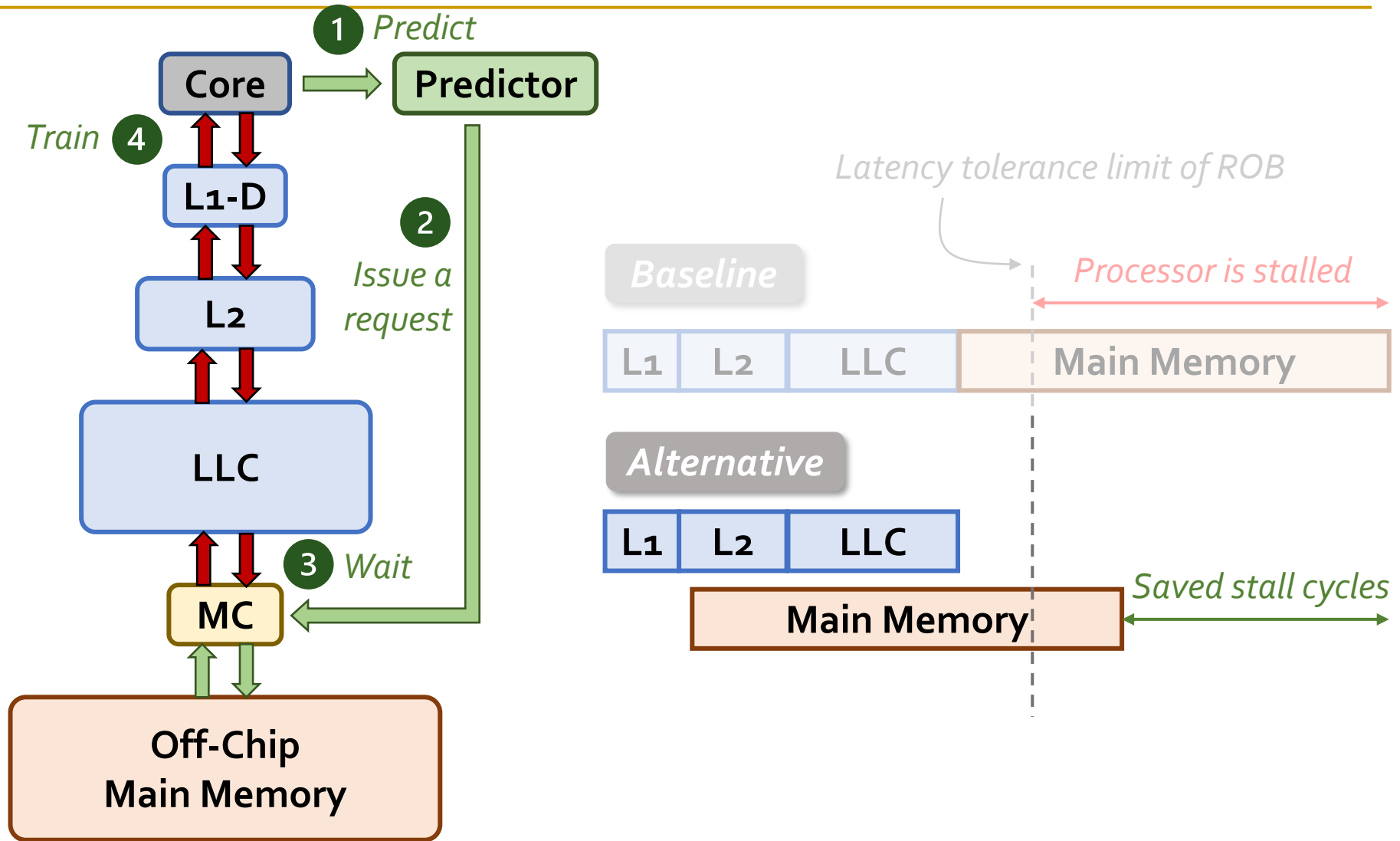
A Case for MLP-Aware Cache Replacement

Moinuddin K. Qureshi Daniel N. Lynch Onur Mutlu Yale N. Patt
Department of Electrical and Computer Engineering
The University of Texas at Austin
{moin, lynch, onur, patt}@hps.utexas.edu

Another form of Parallelism in Memory Access



Another form of Parallelism in Memory Access



Optional Reading: Off-Chip Prediction

- How to identify which memory requests would miss cache hierarchy?
- How to exploit the observation to reduce memory access latency?

Best paper award at MICRO 2022.



Hermes: Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction

Rahul Bera¹ Konstantinos Kanellopoulos¹ Shankar Balachandran² David Novo³
Ataberk Olgun¹ Mohammad Sadrosadati¹ Onur Mutlu¹

¹ETH Zürich ²Intel Processor Architecture Research Lab ³LIRMM, Univ. Montpellier, CNRS

Learning-Based Off-Chip Load Prediction

- Rahul Bera, Konstantinos Kanellopoulos, Shankar Balachandran, David Novo, Ataberk Olgun, Mohammad Sadrosadati, and Onur Mutlu,
"Hermes: Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction"

Proceedings of the 55th International Symposium on Microarchitecture (MICRO), Chicago, IL, USA, October 2022.

[[Slides \(pptx\)](#) ([pdf](#))]

[[Longer Lecture Slides \(pptx\)](#) ([pdf](#))]

[[Talk Video](#) (12 minutes)]

[[Lecture Video](#) (25 minutes)]

[[arXiv version](#)]

[[Source Code \(Officially Artifact Evaluated with All Badges\)](#)]

Officially artifact evaluated as available, reusable and reproducible.

Best paper award at MICRO 2022.



Hermes: Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction

Rahul Bera¹ Konstantinos Kanellopoulos¹ Shankar Balachandran² David Novo³
Ataberk Olgun¹ Mohammad Sadrosadati¹ Onur Mutlu¹

¹ETH Zürich ²Intel Processor Architecture Research Lab ³LIRMM, Univ. Montpellier, CNRS

<https://arxiv.org/pdf/2209.00188.pdf>

Optional Reading: Off-Chip Prediction

- Rahul Bera, Zhenrong Lang, Caroline Hengartner, Konstantinos Kanellopoulos, Rakesh Kumar, Mohammad Sadrosadati, and Onur Mutlu,

"Athena: Synergizing Data Prefetching and Off-Chip Prediction via Online Reinforcement Learning"

Proceedings of the 32nd International Symposium on High-Performance Computer Architecture (HPCA), Sydney, Australia, February 2026.

[Slides (pptx) (pdf)]

[Athena Source Code (Officially Artifact Evaluated with All Badges)]

***Officially artifact evaluated as available, functional, and reproduced.
Distinguished artifact award at HPCA 2026.***



Athena: Synergizing Data Prefetching and Off-Chip Prediction via Online Reinforcement Learning

*Rahul Bera¹ *Zhenrong Lang¹ Caroline Hengartner¹ Konstantinos Kanellopoulos¹
Rakesh Kumar² Mohammad Sadrosadati¹ Onur Mutlu¹

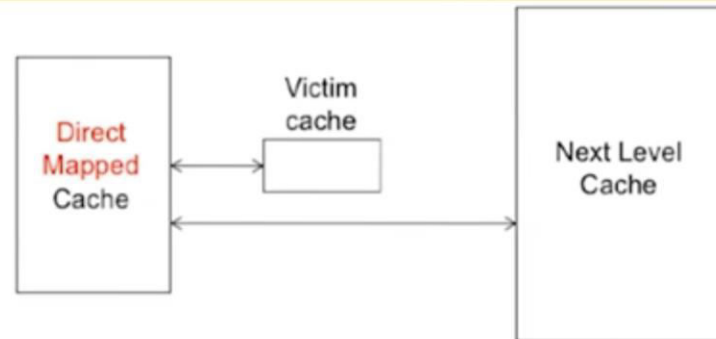
¹ETH Zürich ²NTNU

Improving Basic Cache Performance

- Reducing miss rate
 - ❑ More associativity
 - ❑ Alternatives/enhancements to associativity
 - Victim caches, hashing, pseudo-associativity, skewed associativity
 - ❑ Better replacement/insertion policies
 - ❑ Software approaches
 - ❑ ...
- Reducing miss latency/cost
 - ❑ Multi-level caches
 - ❑ Critical word first
 - ❑ Subblocking/sectoring
 - ❑ Better replacement/insertion policies
 - ❑ Non-blocking caches (multiple cache misses in parallel)
 - ❑ Multiple accesses per cycle
 - ❑ Software approaches
 - ❑ ...

Lectures on Cache Optimizations (I)

Victim Cache: Reducing Conflict Misses



- Jouppi, "Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers," ISCA 1990.
- Idea: Use a small fully-associative buffer (victim cache) to store recently evicted blocks
 - + Can avoid ping ponging of cache blocks mapped to the same set (if two cache blocks continuously accessed in nearby time conflict with each other)
 - Increases miss latency if accessed serially with L2; adds complexity

Computer Architecture - Lecture 3: Cache Management and Memory Parallelism (ETH Zürich, Fall 2017)

6,392 views • Sep 29, 2017

49 1 SHARE SAVE ...

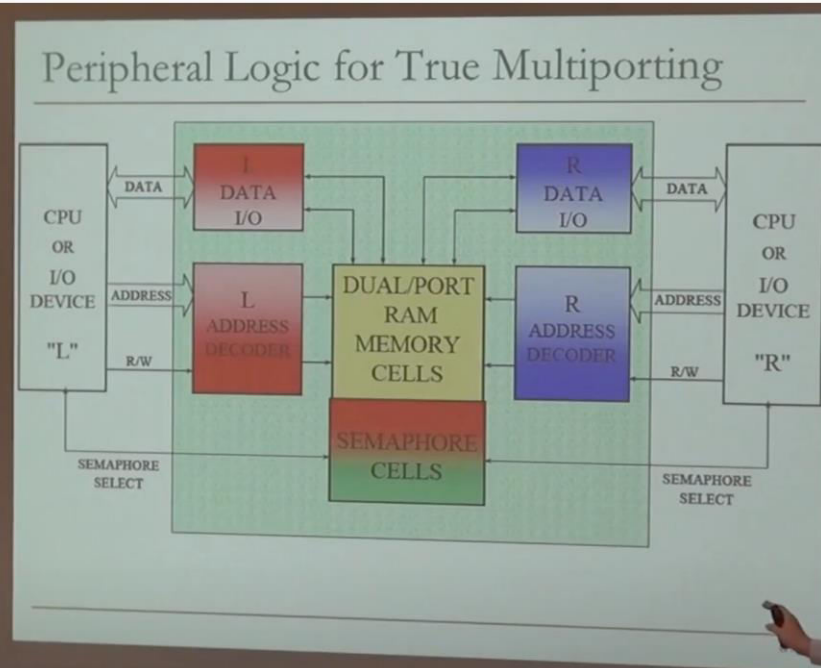


Onur Mutlu Lectures
16.3K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Cache Optimizations (II)



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 4a: Cache Design (ETH Zürich, Fall 2018)

1,437 views • Sep 29, 2018

15 0 SHARE SAVE ...



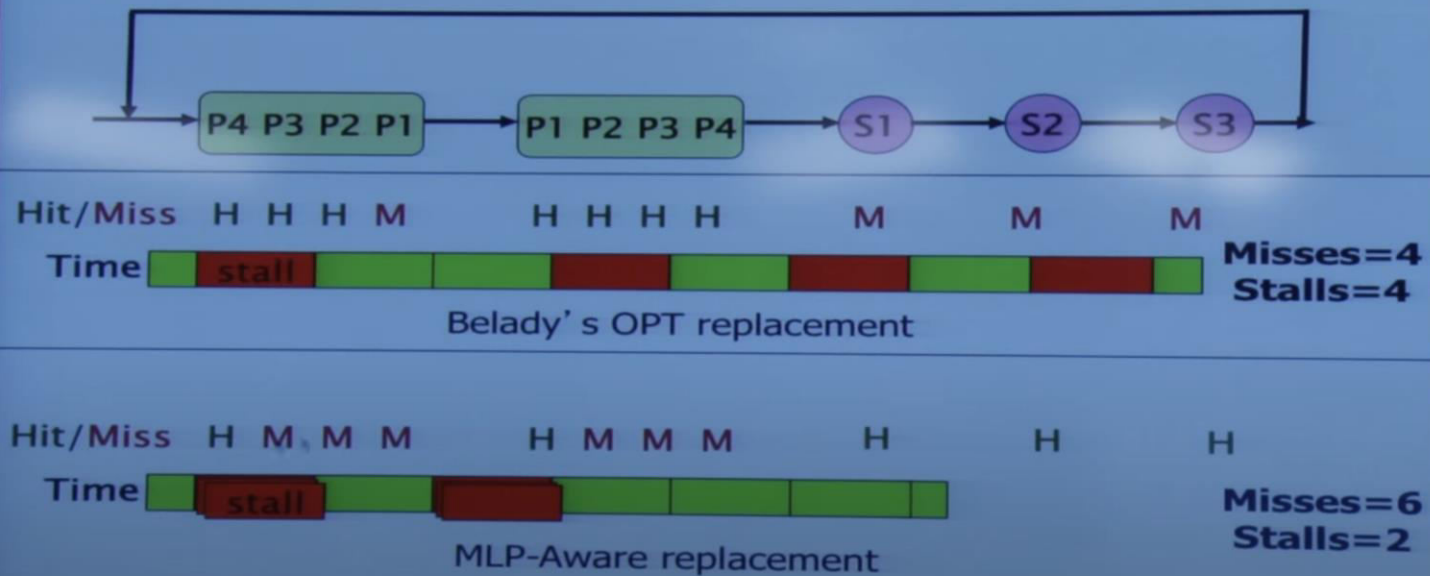
Onur Mutlu Lectures
16.3K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Cache Optimizations (III)

Fewest Misses $\neq$ Best Performance



Lecture 19. High Performance Caches - Carnegie Mellon - Comp. Arch. 2015 - Onur Mutlu

9,737 views • Mar 5, 2015

63 1 SHARE SAVE ...



Carnegie Mellon Computer Architecture
23.2K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Cache Optimizations

■ Computer Architecture, Fall 2017, Lecture 3

- ❑ Cache Management & Memory Parallelism (ETH, Fall 2017)
- ❑ <https://www.youtube.com/watch?v=OyomXCHNJDA&list=PL5Q2soXY2Zi9OhoVQBX YFIZywZXCPI4M &index=3>

■ Computer Architecture, Fall 2018, Lecture 4a

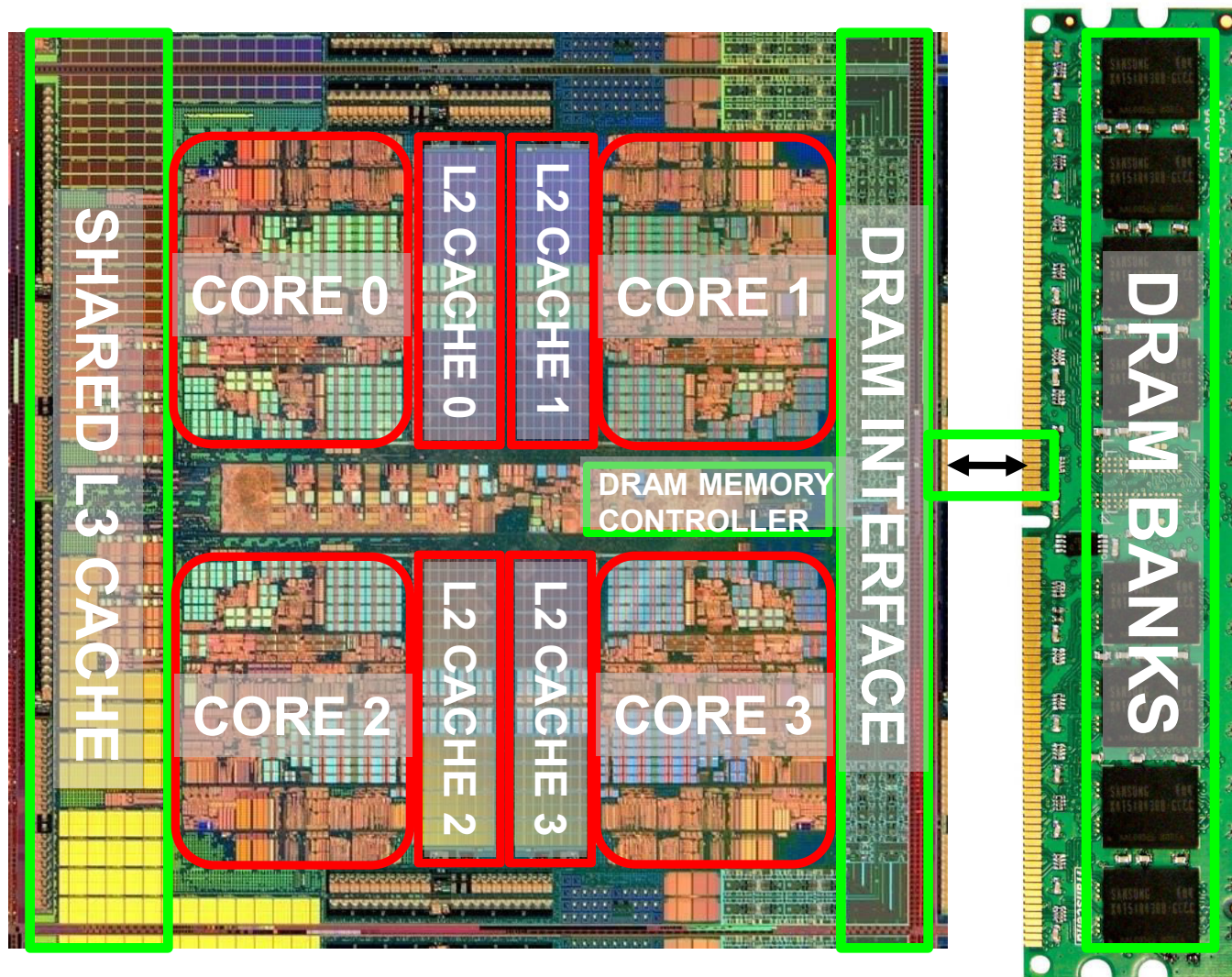
- ❑ Cache Design (ETH, Fall 2018)
- ❑ https://www.youtube.com/watch?v=55oYBm9cifI&list=PL5Q2soXY2Zi9JXe3ywQMh ylk_d5dI-TM7&index=6

■ Computer Architecture, Spring 2015, Lecture 19

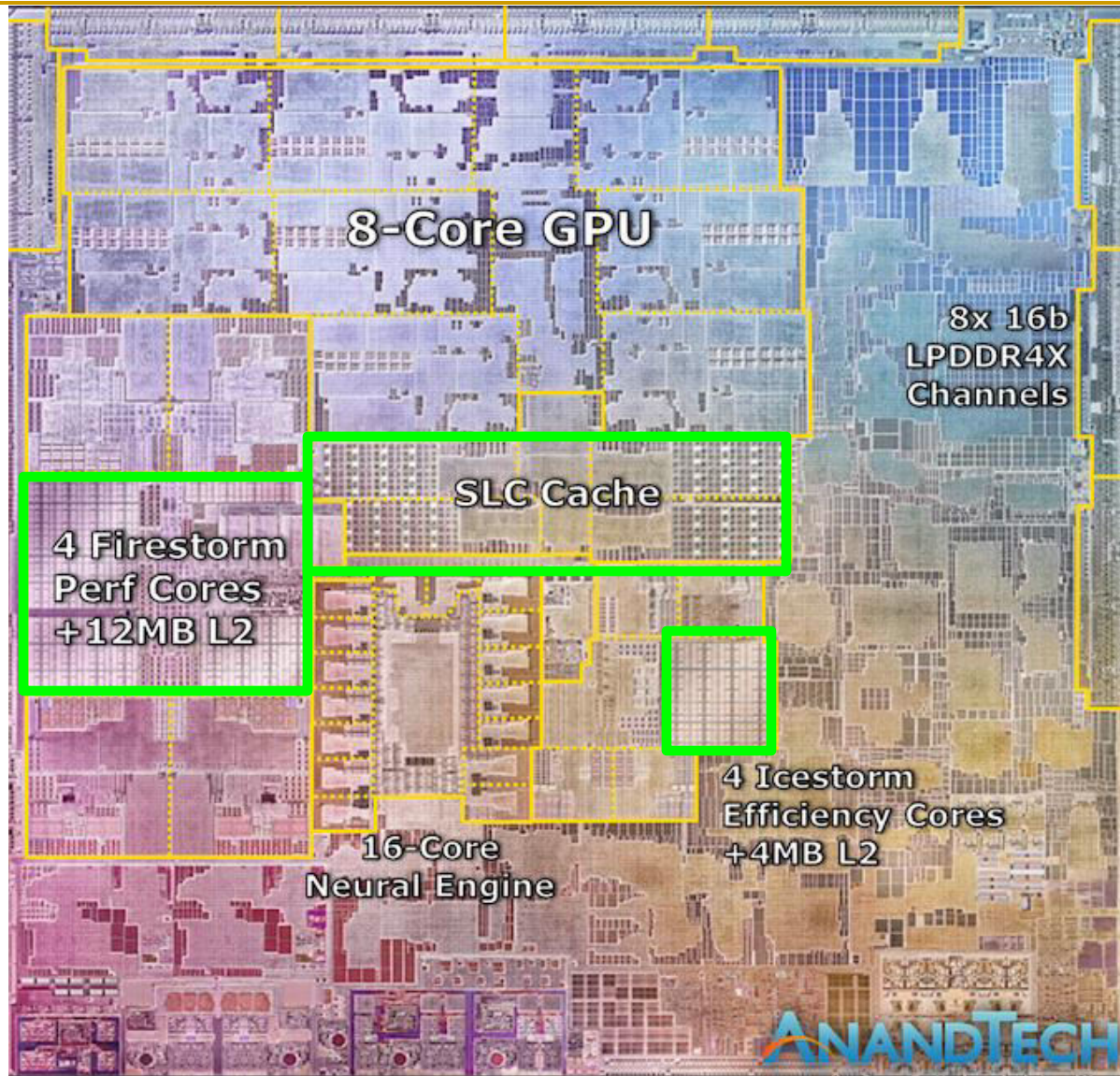
- ❑ High Performance Caches (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=jDHx2K9HxIM&list=PL5PHm2jkkXmi5CxxI7b3J CL1TWybTDtKq&index=21>

Multi-Core Issues in Caching

Caches in a Multi-Core System



Caches in a Multi-Core System



Apple M1,
2021

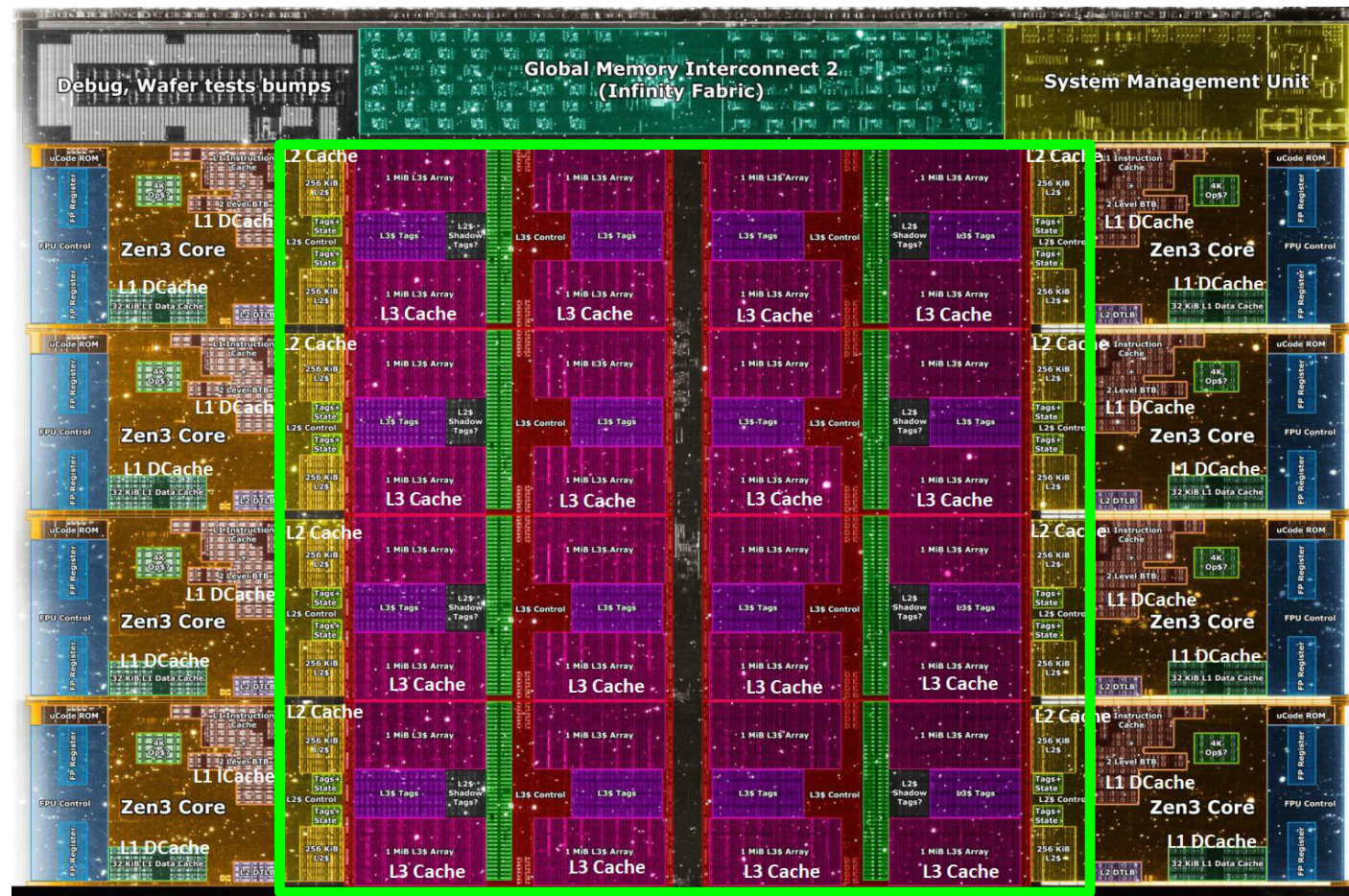
Caches in a Multi-Core System



10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>
Die shot interpretation by Locuza, October 2021

Intel Alder Lake,
2021

Caches in a Multi-Core System



Core Count:

8 cores/16 threads

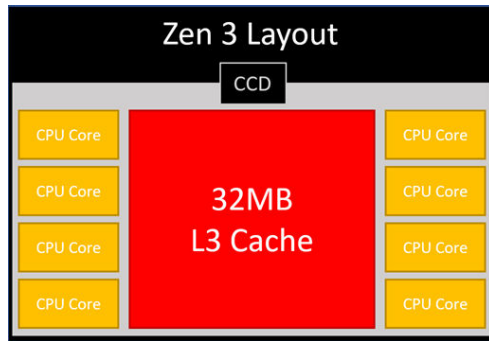
L1 Caches:
32 KB per core

L2 Caches:
512 KB per core

L3 Cache:
32 MB shared

AMD Ryzen 5000, 2020

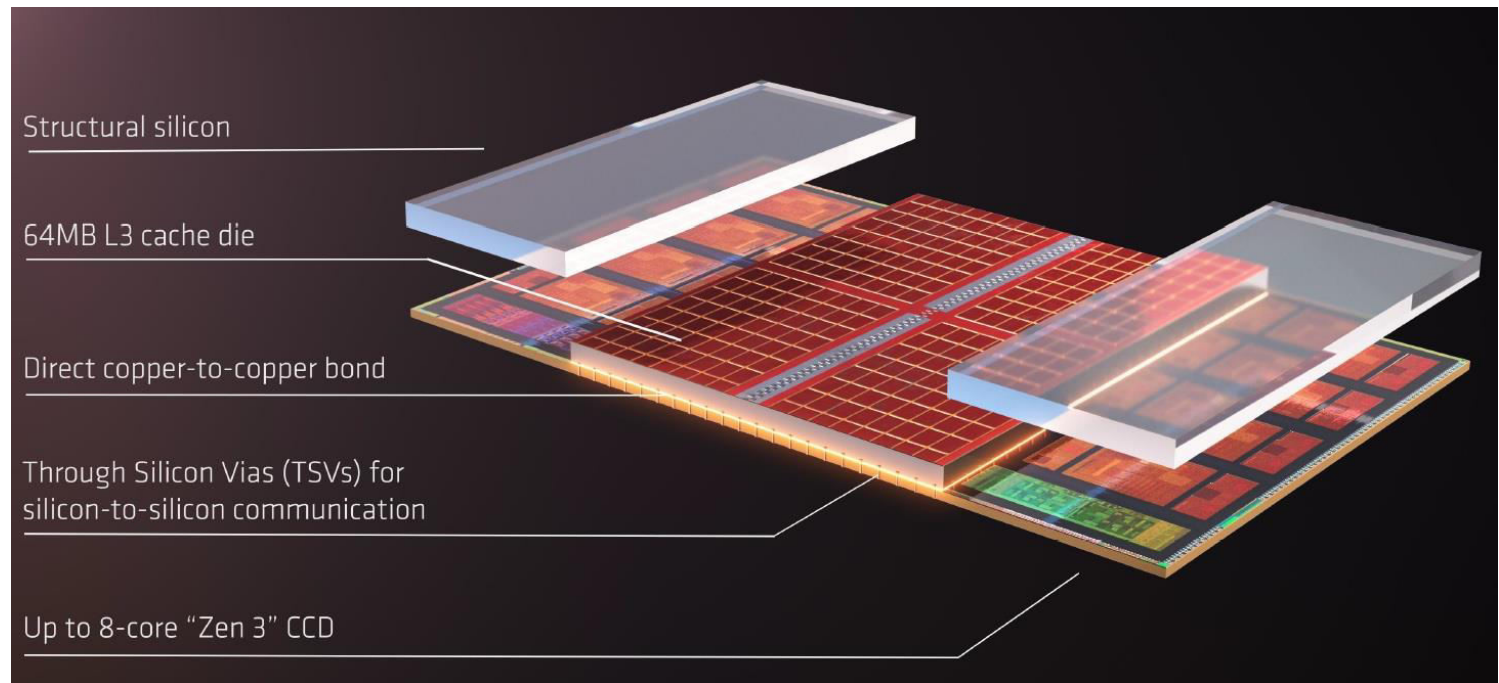
Caches in a Multi-Core System



<https://community.microcenter.com/discussion/5134/comparing-zen-3-to-zen-2>

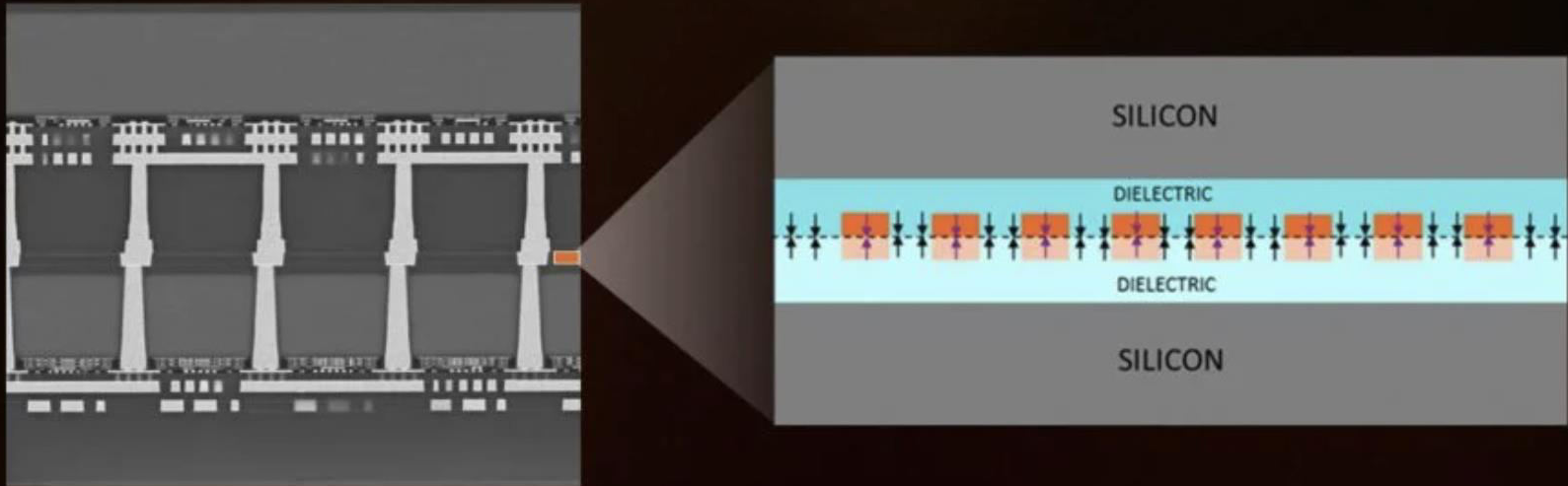
AMD increases the L3 size of their 8-core Zen 3 processors from 32 MB to 96 MB

- Additional 64 MB L3 cache die**
stacked on top of the processor die
- Connected using Through Silicon Vias (TSVs)
 - Total of 96 MB L3 cache



3D Stacking Technology: Example

AMD 3D V-CACHE™ HYBRID BONDING

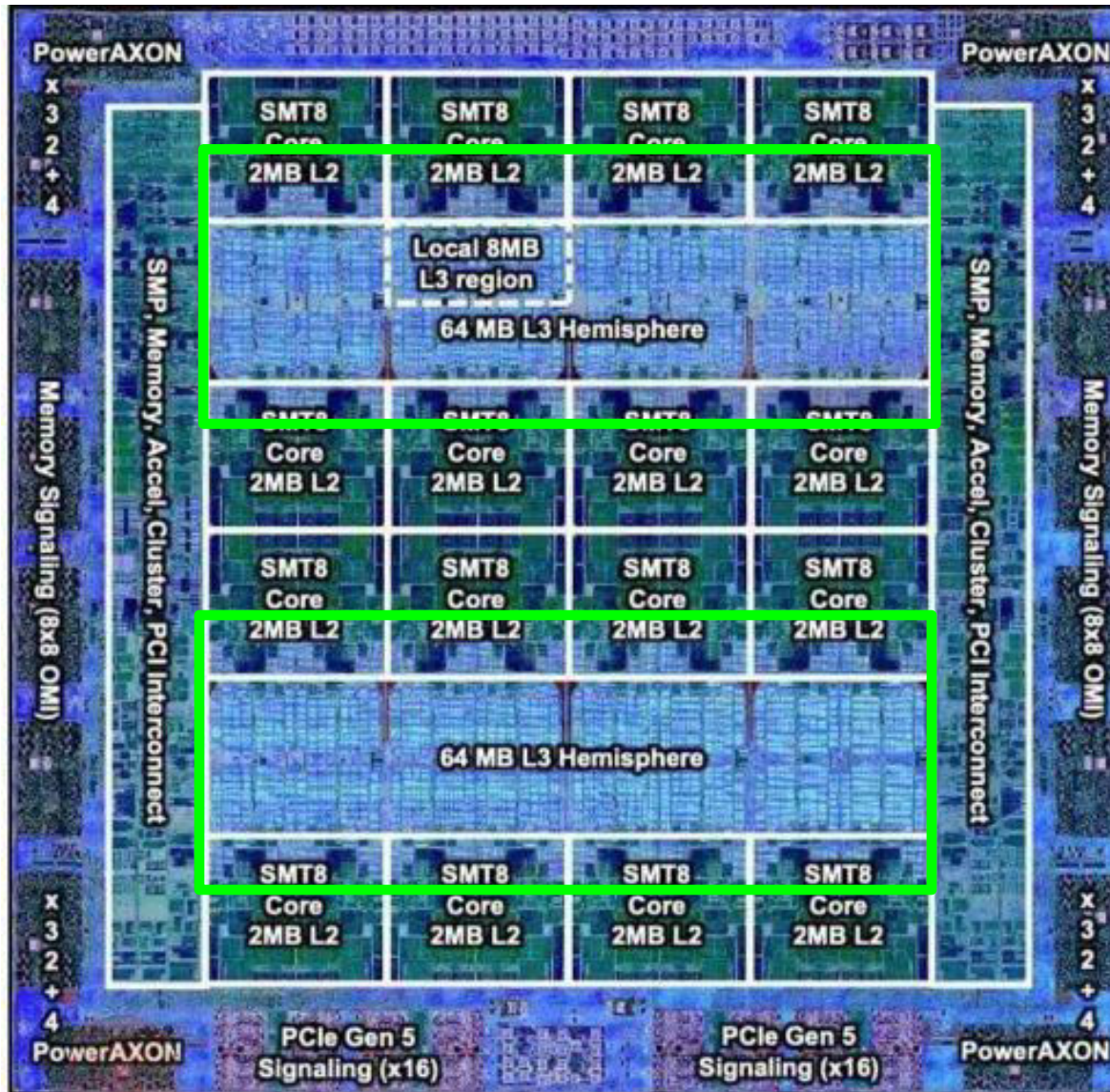


Direct Copper – Copper Bonding

AMD Ryzen 7 5800X3D: The 3D V-Cache in detail (4)

Source: AMD

Caches in a Multi-Core System



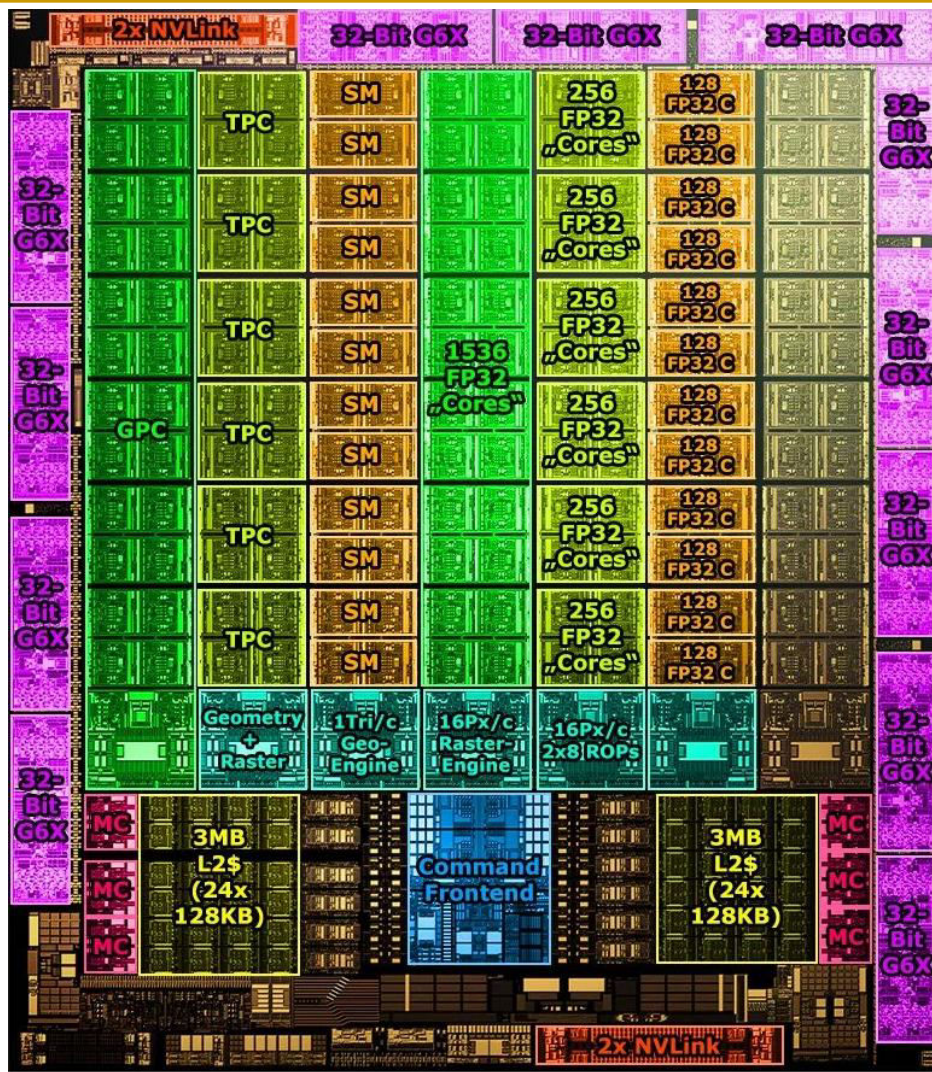
IBM POWER10,
2020

Cores:
15-16 cores,
8 threads/core

L2 Caches:
2 MB per core

L3 Cache:
120 MB shared

Caches in a Multi-Core System



Nvidia Ampere, 2020

Cores:

128 Streaming Multiprocessors

L1 Cache or Scratchpad:

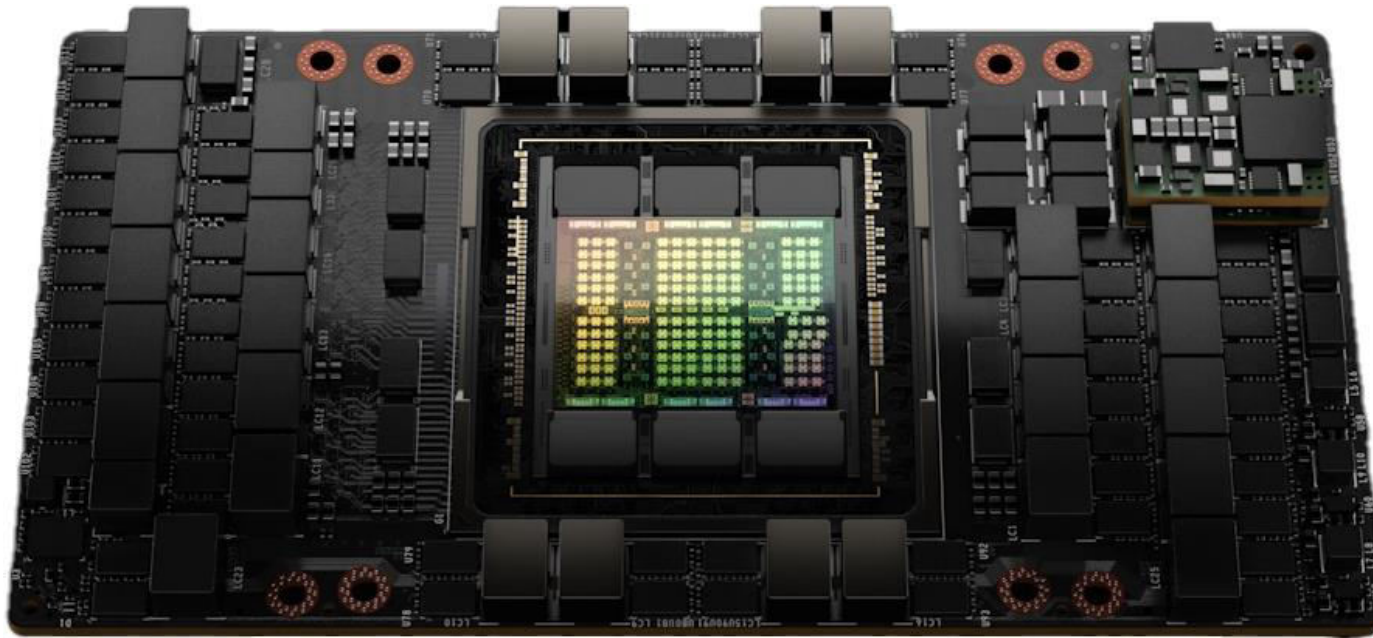
192KB per SM

Can be used as L1 Cache and/or Scratchpad

L2 Cache:

40 MB shared

Caches in a Multi-Core System



Nvidia Hopper,
2022

Cores:
144 Streaming
Multiprocessors

**L1 Cache or
Scratchpad:**
256KB per SM
Can be used as L1 Cache
and/or Scratchpad

L2 Cache:
60 MB shared

Caches in Multi-Core Systems

- Cache efficiency becomes even more important in a multi-core/multi-threaded system
 - Memory bandwidth is at premium
 - Cache space is a limited resource across cores/threads
- How do we design the caches in a multi-core system?
- Many decisions and questions
 - Shared vs. private caches
 - How to maximize performance of the entire system?
 - How to provide QoS & predictable perf. to different threads in a shared cache?
 - Should cache management algorithms be aware of threads?
 - How should space be allocated to threads in a shared cache?
 - Should we store data in compressed format in some caches?
 - How do we do better reuse prediction & management in caches?

Compressed Caches

- Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Philip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry,
"Base-Delta-Immediate Compression: Practical Data Compression for On-Chip Caches"
Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT), Minneapolis, MN, September 2012. [Slides \(pptx\)](#) [Source Code](#)

Base-Delta-Immediate Compression: Practical Data Compression for On-Chip Caches

Gennady Pekhimenko[†]
gpekhime@cs.cmu.edu

Vivek Seshadri[†]
vseshadr@cs.cmu.edu

Onur Mutlu[†]
onur@cmu.edu

Michael A. Kozuch^{*}
michael.a.kozuch@intel.com

Phillip B. Gibbons^{*}
phillip.b.gibbons@intel.com

Todd C. Mowry[†]
tcm@cs.cmu.edu

[†]Carnegie Mellon University

^{*}Intel Labs Pittsburgh

Key Data Patterns in Real Applications

Zero Values: initialization, sparse matrices, NULL pointers

0x00000000	0x00000000	0x00000000	0x00000000	...
------------	------------	------------	------------	-----

Repeated Values: common initial values, adjacent pixels

0x000000FF	0x000000FF	0x000000FF	0x000000FF	...
------------	------------	------------	------------	-----

Narrow Values: small values stored in a big data type

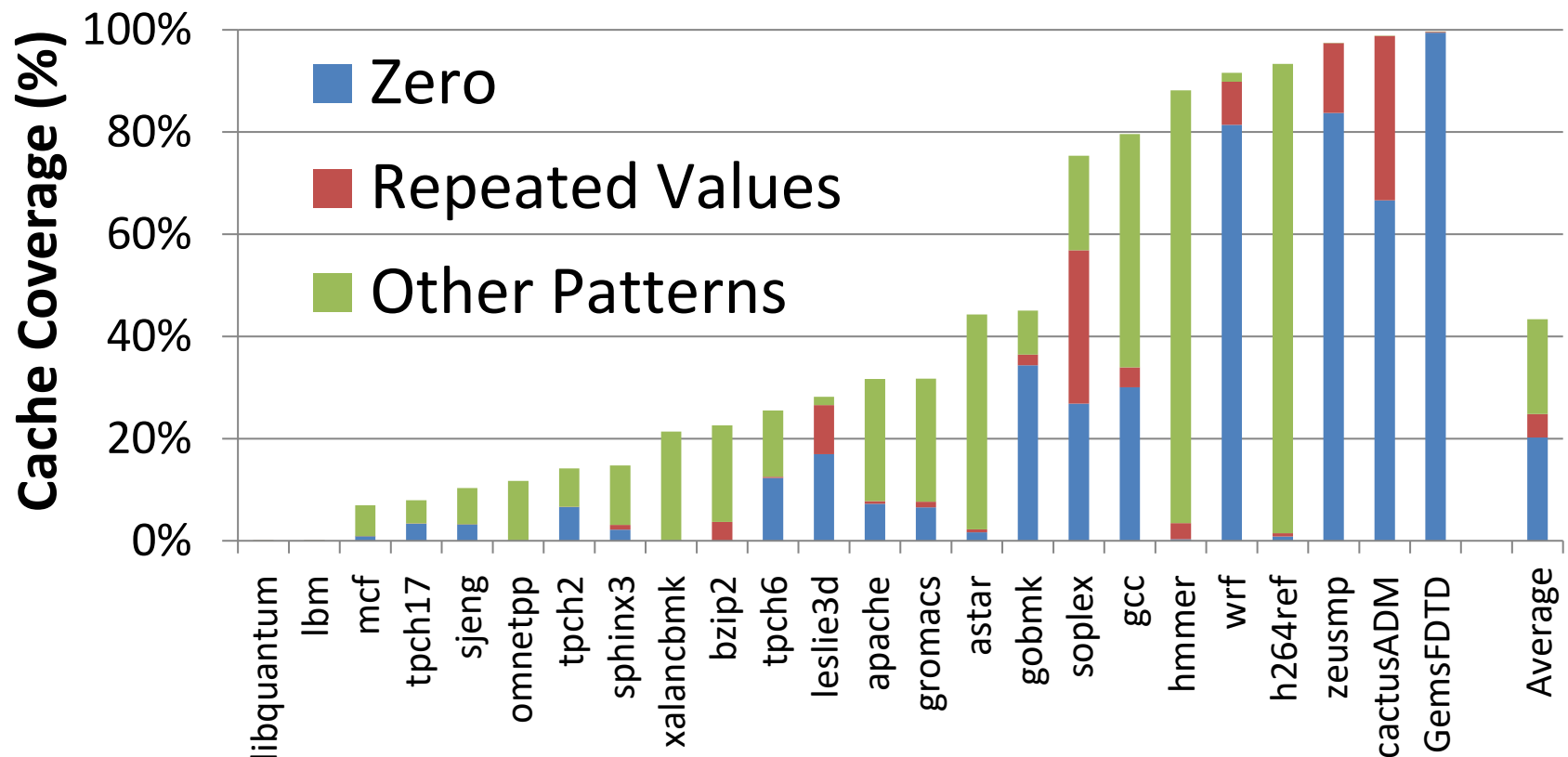
0x00000000	0x0000000B	0x00000003	0x00000004	...
------------	------------	------------	------------	-----

Other Patterns: pointers to the same memory region

0xC04039C0	0xC04039C8	0xC04039D0	0xC04039D8	...
------------	------------	------------	------------	-----

How Common Are These Patterns?

SPEC2006, databases, web workloads, 2MB L2 cache
“Other Patterns” include Narrow Values



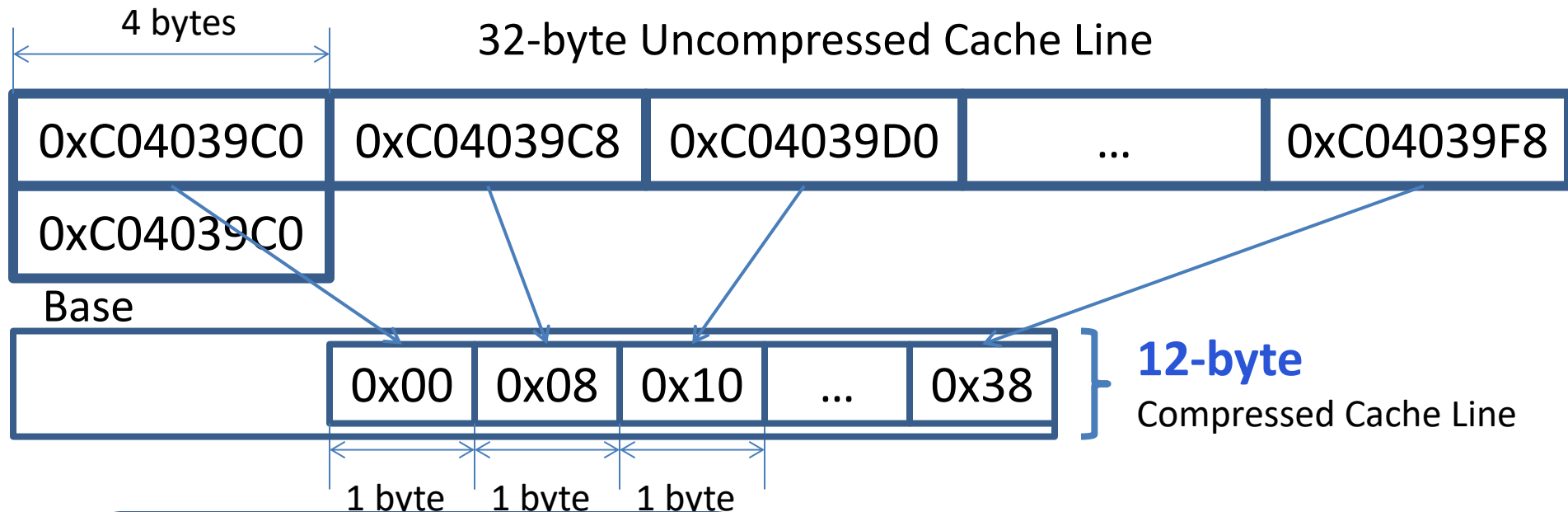
43% of the cache lines belong to key patterns

Key Data Patterns in Real Applications

Low Dynamic Range:

Differences between values are significantly smaller than the values themselves

Key Idea: Base+Delta ($B+\Delta$) Encoding



✓ **Fast Decompression:**
vector addition

✓ **Simple Hardware:**
arithmetic and comparison

✓ **Effective:** good compression ratio

Can We Do Better?

- Uncompressible cache line (with a single base):

0x00000000	0x09A40178	0x0000000B	0x09A4A838	...
------------	------------	------------	------------	-----

- **Key idea:**
Use more bases, e.g., two instead of one
- **Pro:**
 - More cache lines can be compressed
- **Cons:**
 - Unclear how to find these bases efficiently
 - Higher overhead (due to additional bases)

How to Find Two Bases Efficiently?

1. **First base** - first element in the cache line

✓ **Base+Delta part**

2. **Second base** - implicit base of 0

✓ **Immediate part**

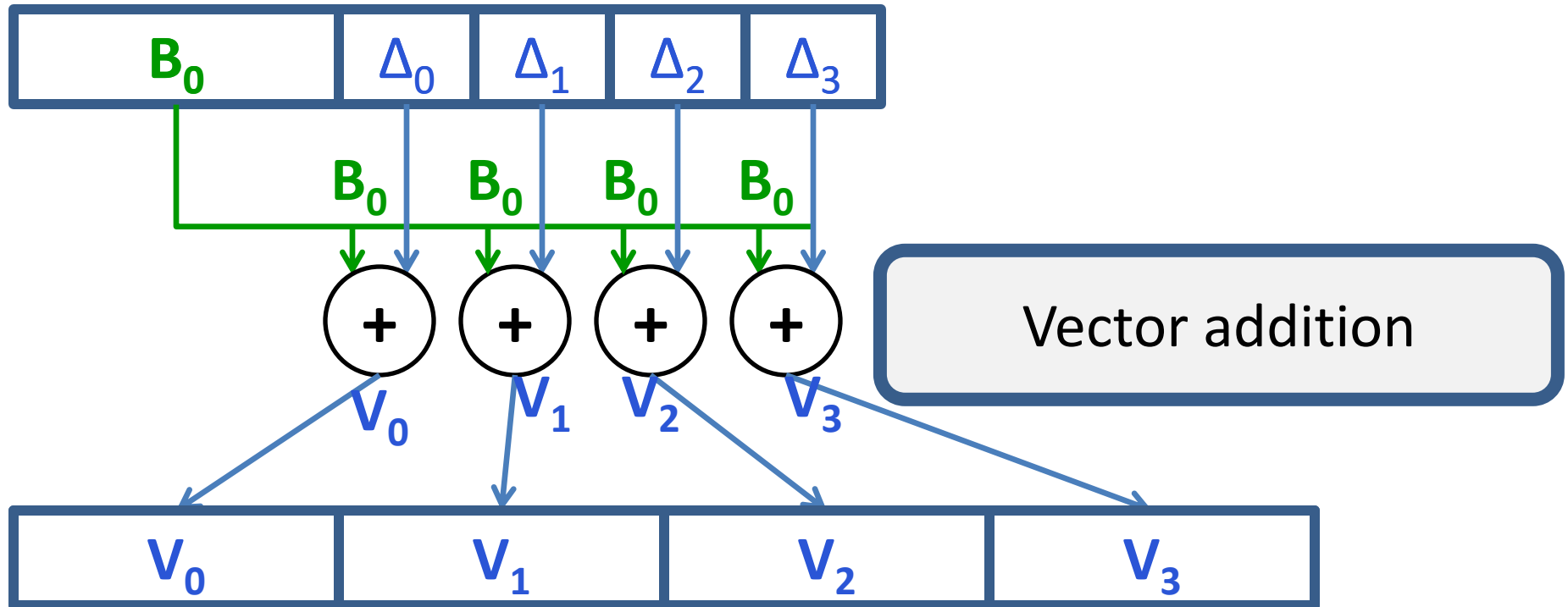
Advantages over 2 arbitrary bases:

- Better compression ratio
- Simpler compression logic

Base-Delta-Immediate (BΔI) Compression

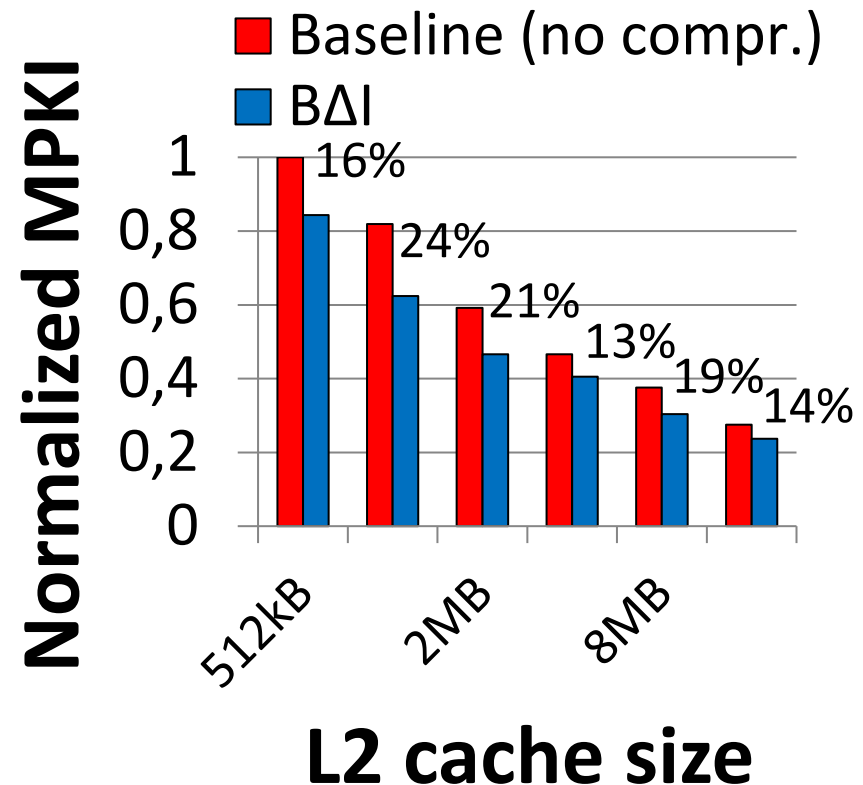
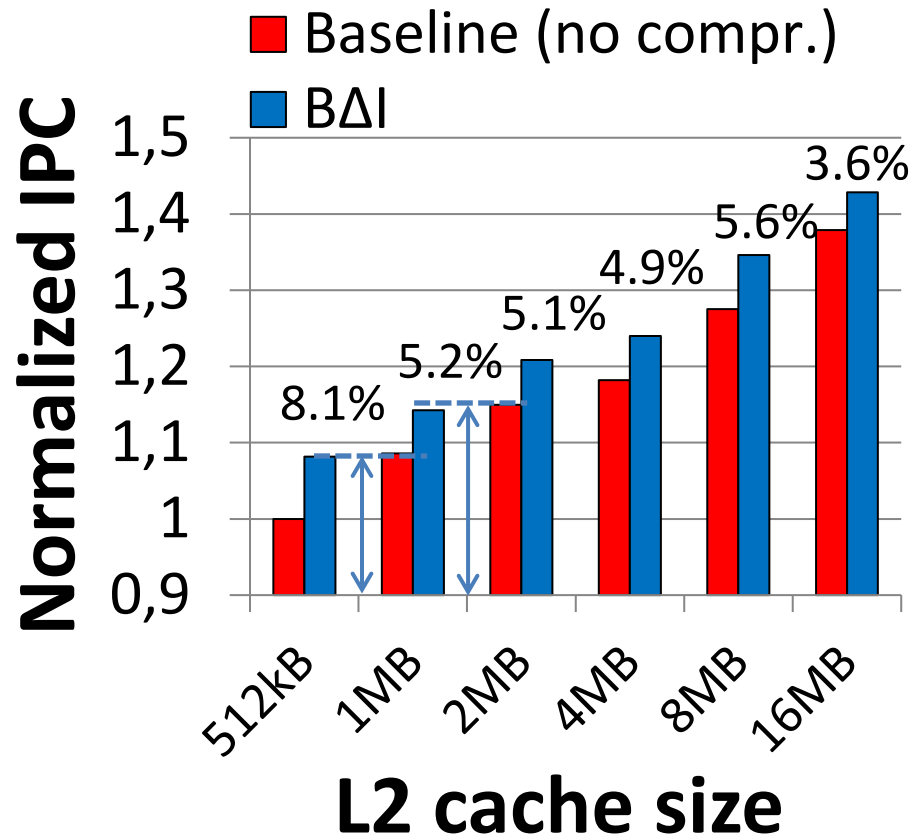
B Δ I Decompressor Design

Compressed Cache Line



Uncompressed Cache Line

Performance



BΔI achieves the performance of a 2X-size cache

Performance improves due to the decrease in MPKI

Base-Delta-Immediate Compression

- Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Philip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry,
"Base-Delta-Immediate Compression: Practical Data Compression for On-Chip Caches"
Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT), Minneapolis, MN, September 2012. [Slides \(pptx\)](#) [Source Code](#)

Base-Delta-Immediate Compression: Practical Data Compression for On-Chip Caches

Gennady Pekhimenko[†]
gpekhime@cs.cmu.edu

Vivek Seshadri[†]
vseshadr@cs.cmu.edu

Onur Mutlu[†]
onur@cmu.edu

Michael A. Kozuch^{*}
michael.a.kozuch@intel.com

Phillip B. Gibbons^{*}
phillip.b.gibbons@intel.com

Todd C. Mowry[†]
tcm@cs.cmu.edu

[†]Carnegie Mellon University

^{*}Intel Labs Pittsburgh

Cache Compression in Real Systems

Enterprise-Class Cache Compression Design

Alper Buyuktosunoglu[†] David Trilla[†] Bulent Abali[†] Deanna Berger[‡] Craig Walters[‡] Jang-Soo Lee[‡]
IBM Research[†] IBM Infrastructure[‡]

Enterprise Class Modular Cache Hierarchy

Craig Walters[‡] Deanna Berger[†] Robert Sonnelitter[†] Alper Buyuktosunoglu[‡]
IBM Infrastructure[†], IBM Research[‡]

Cache Compression in Huge Caches

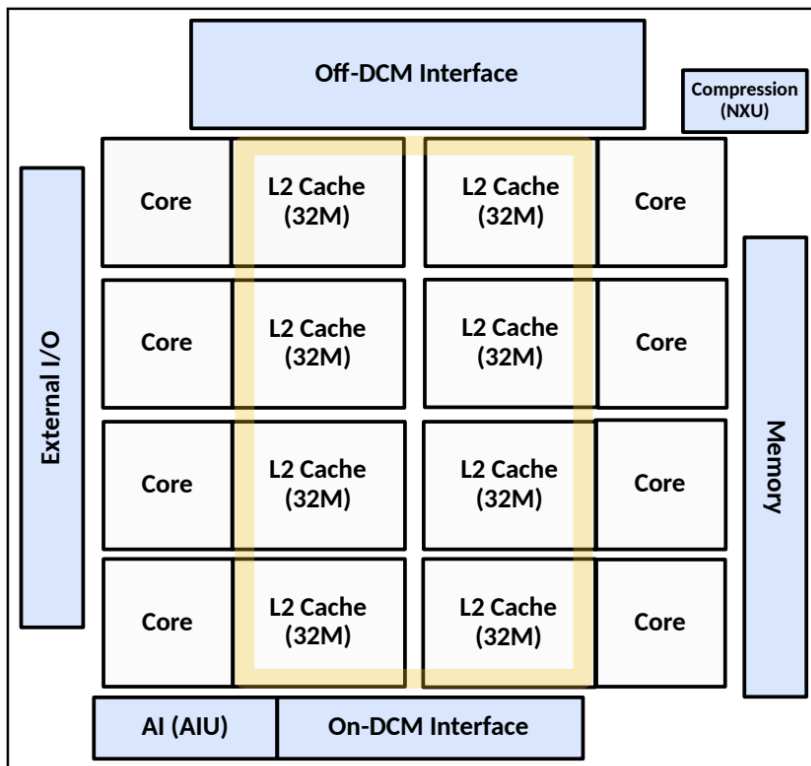


Fig. 1. Telum® Processor Chip

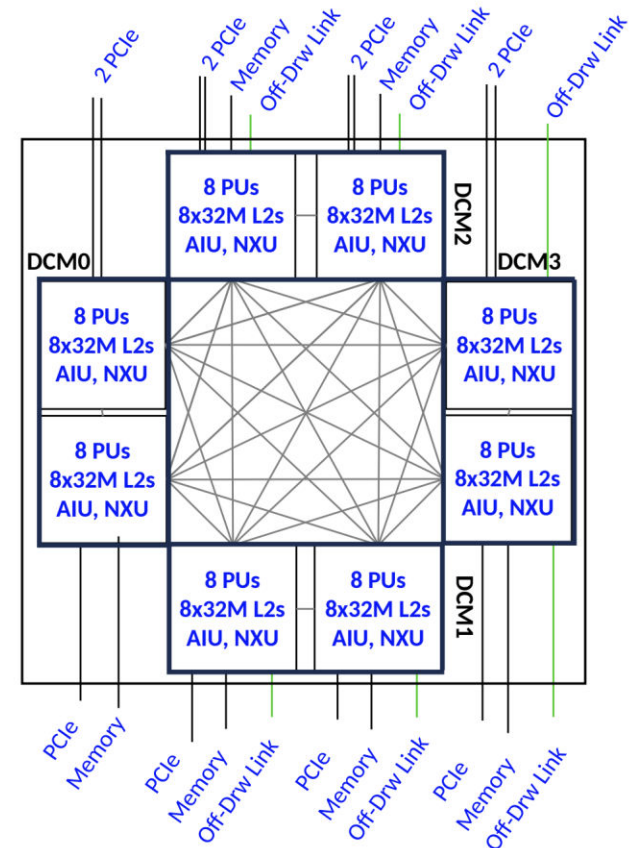


Fig. 2. Telum® Processor Drawer. PUs: Processor Cores. L2s: L2-Caches. AIU: AI Accelerator. NXU: Data Compression Accelerator

Cache Performance w/ Huge Caches

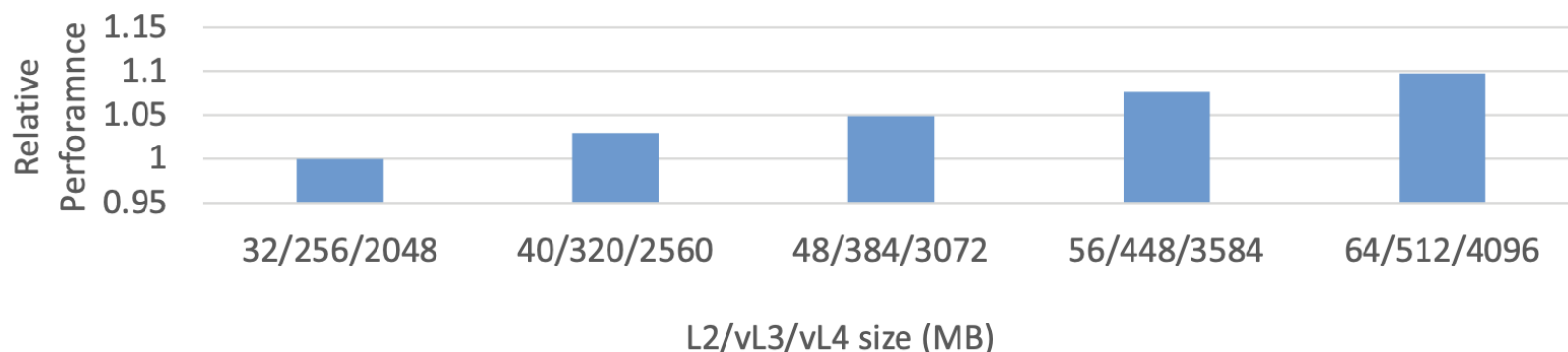
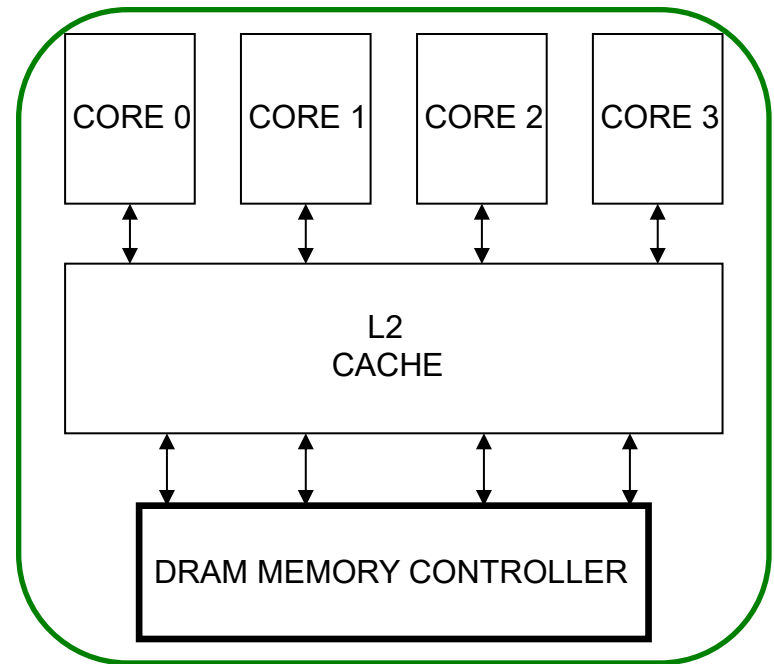
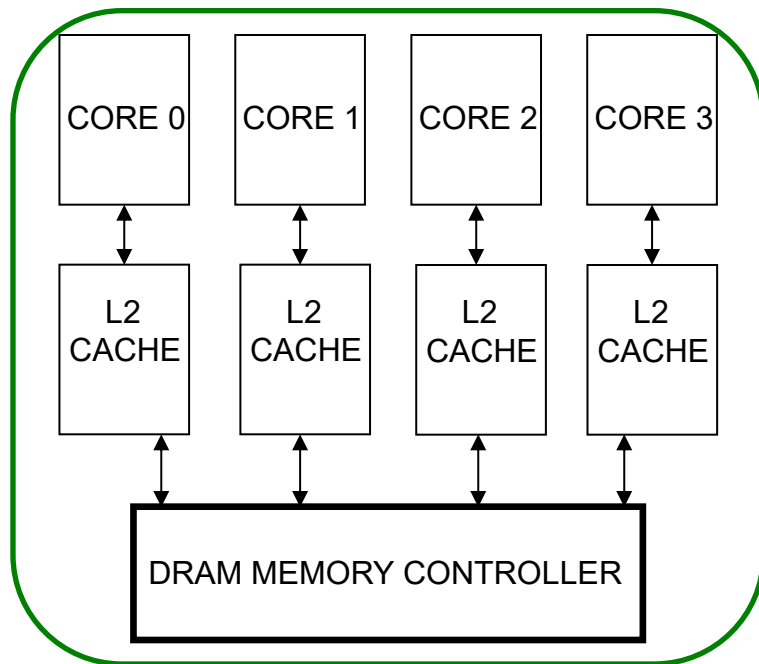


Fig. 1. Relative performance of a 28-way SMP relational database workload with different cache sizes compared to a baseline with 32MB L2, 256MB vL3, 2048MB vL4

Buyuktosunoglu+, “[Enterprise-Class Cache Compression Design](#)”, HPCA 2024.

Private vs. Shared Caches

- **Private** cache: Cache belongs to one core (a shared block can be in multiple caches)
- **Shared** cache: Cache is shared by multiple cores



Resource Sharing Concept and Advantages

- Idea: Instead of dedicating a hardware resource to a hardware context, allow multiple contexts to use it
 - Example resources: functional units, pipeline, caches, buses, memory, interconnects, storage
 - Why?
-
- + Resource sharing improves utilization/efficiency → throughput
 - When a resource is left idle by one thread, another thread can use it; no need to replicate shared data
 - + Reduces communication latency
 - For example, data shared between multiple threads can be kept in the same cache in multithreaded processors
 - + Compatible with the shared memory programming model

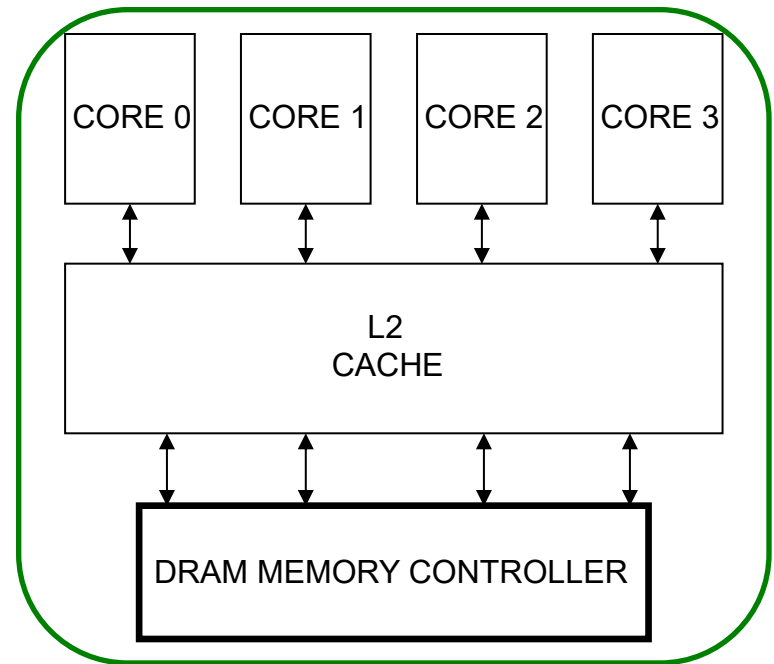
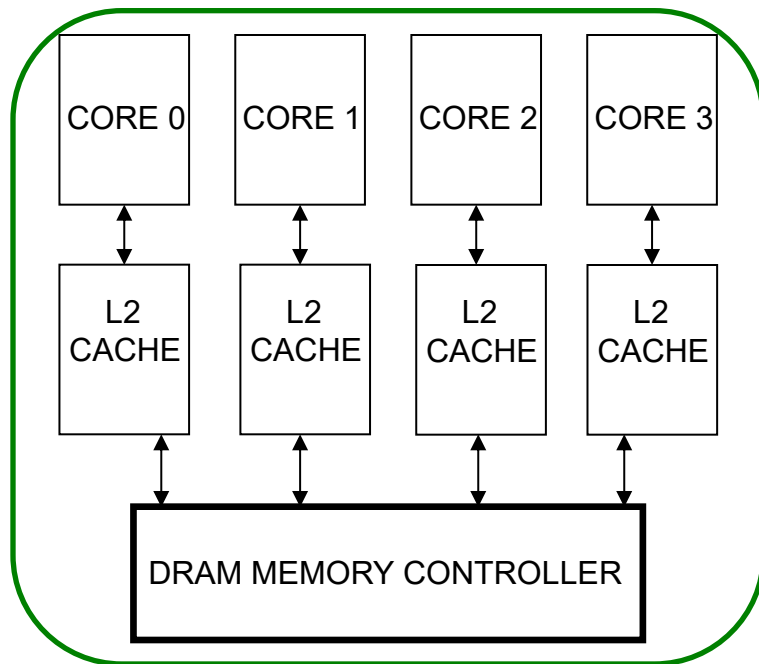
Resource Sharing Disadvantages

- Resource sharing results in **contention for resources**
 - When the resource is not idle, another thread cannot use it
 - If space is occupied by one thread, another thread needs to re-occupy it
- **Sometimes reduces each or some thread's performance**
 - Thread performance can be worse than when it is run alone
- **Eliminates performance isolation** → inconsistent performance across runs
 - Thread performance depends on co-executing threads
- Uncontrolled (free-for-all) sharing **degrades quality of service**
 - Causes unfairness, starvation

Need to efficiently and fairly utilize shared resources

Private vs. Shared Caches

- **Private** cache: Cache belongs to one core (a shared block can be in multiple caches)
- **Shared** cache: Cache is shared by multiple cores



Shared Caches Between Cores

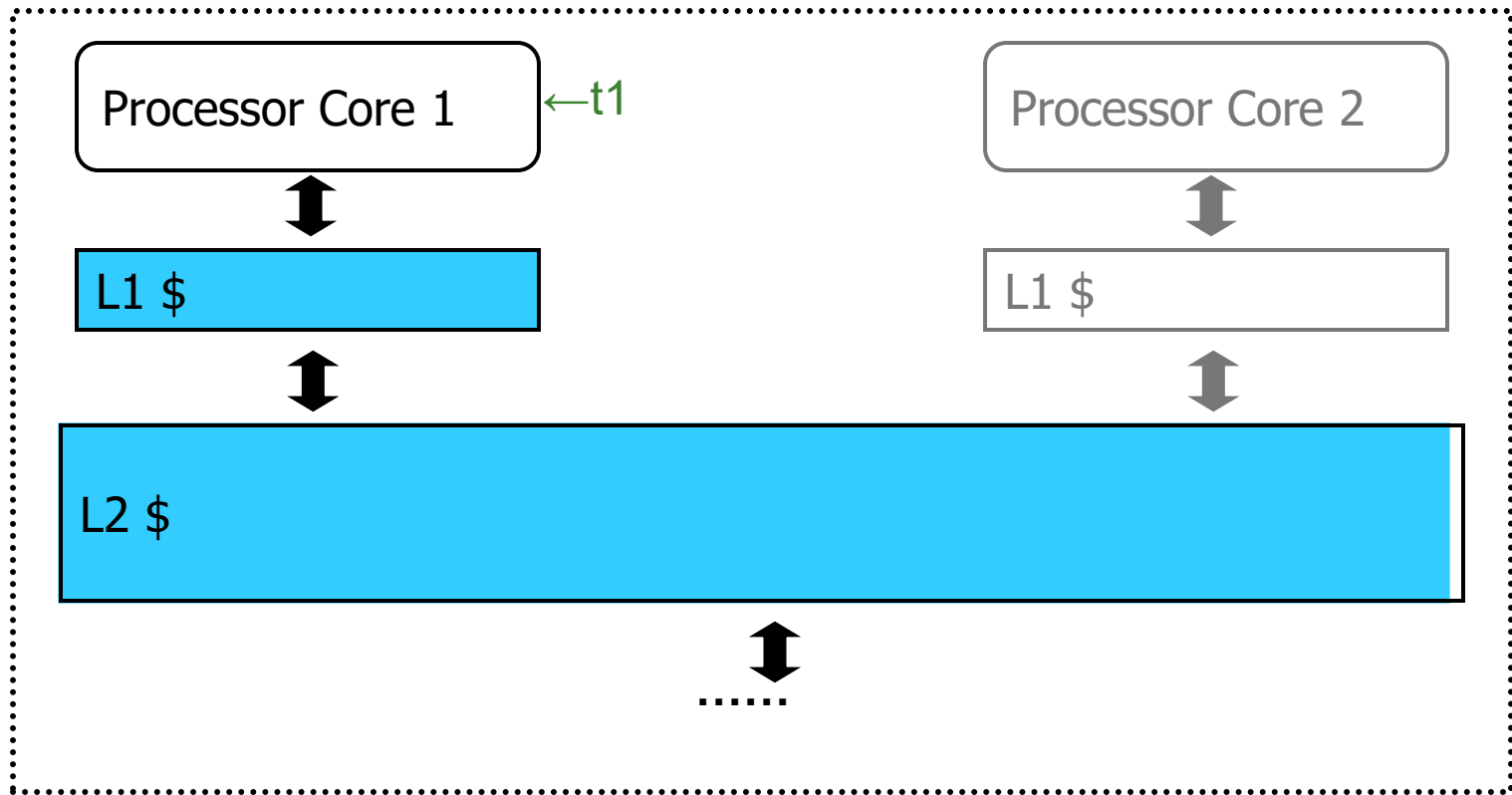
■ Advantages:

- High effective capacity
- Dynamic partitioning of available cache space
 - No fragmentation due to static partitioning
 - If one core does not utilize some space, another core can
- Easier to maintain coherence (a cache block is in a single location)

■ Disadvantages

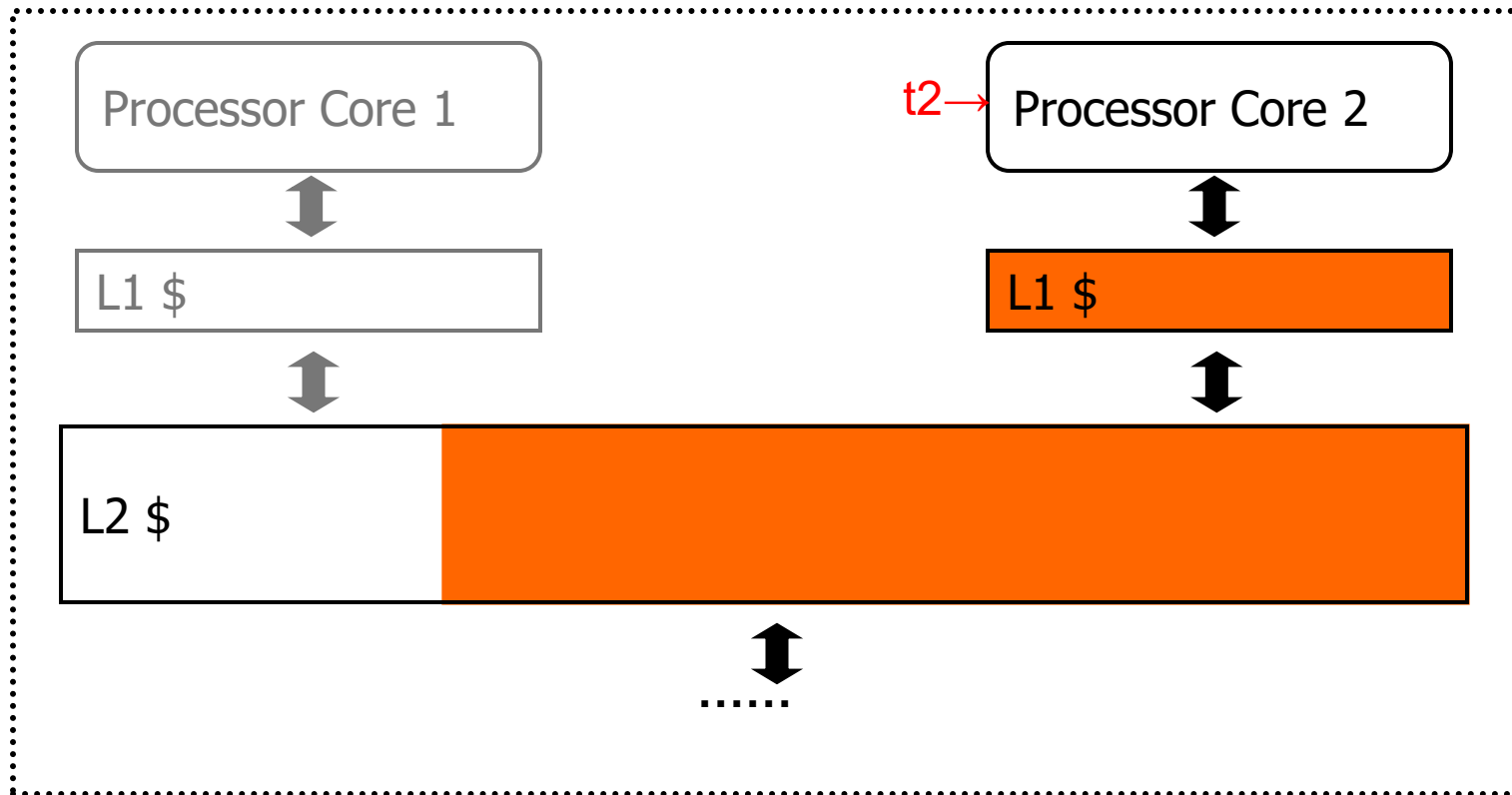
- Slower access (cache is not tightly coupled with the core)
- Cores incur conflict misses due to other cores' accesses
 - Misses due to inter-core interference
 - Some cores can destroy the hit rate of other cores
- Guaranteeing a minimum level of service (or fairness) to each core is harder (how much space, how much bandwidth?)

Example: One Problem with Shared Caches



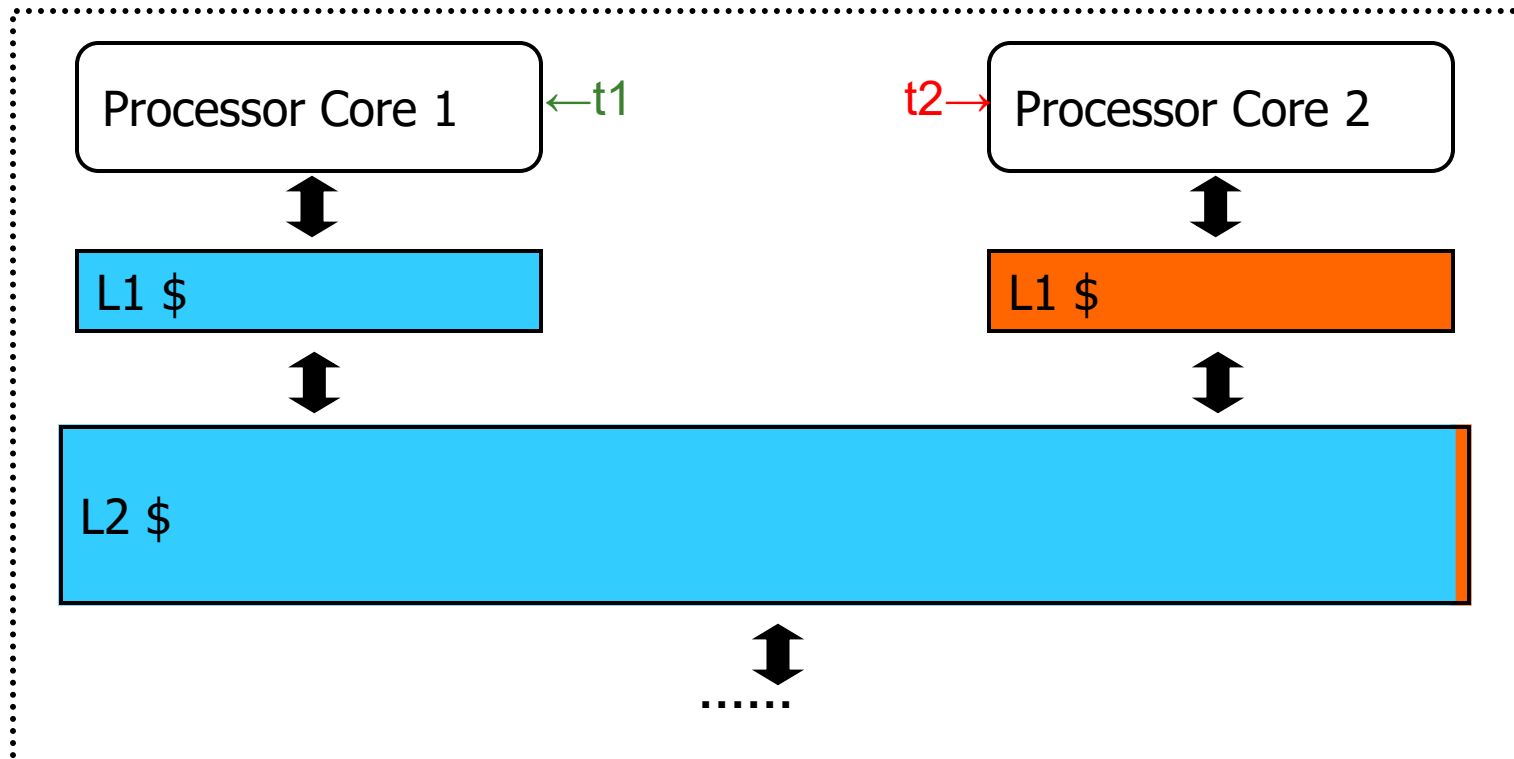
Kim et al., “Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture,” PACT 2004.

Example: One Problem with Shared Caches



Kim et al., “Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture,” PACT 2004.

Example: One Problem with Shared Caches



t2's performance is significantly reduced due to **unfair cache sharing**

Kim et al., "Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture," PACT 2004.

Resource Sharing vs. Partitioning

- Sharing improves resource utilization
 - Better utilization of space
- Partitioning provides performance isolation (predictable performance)
 - Dedicated space
- Can we get the benefits of both?
- Idea: Design shared resources such that they are efficiently utilized, controllable and partitionable
 - No wasted resource + QoS mechanisms for threads

Lectures on Multi-Core Cache Management

Computer Architecture

Lecture 15: Multi-Core Cache Management

Prof. Onur Mutlu
ETH Zürich
Fall 2017
15 November 2017



0:35 / 2:33:03



Computer Architecture - Lecture 15: Multi-Core Cache Management (ETH Zürich, Fall 2017)

934 views • Nov 17, 2017

13 0 SHARE SAVE ...

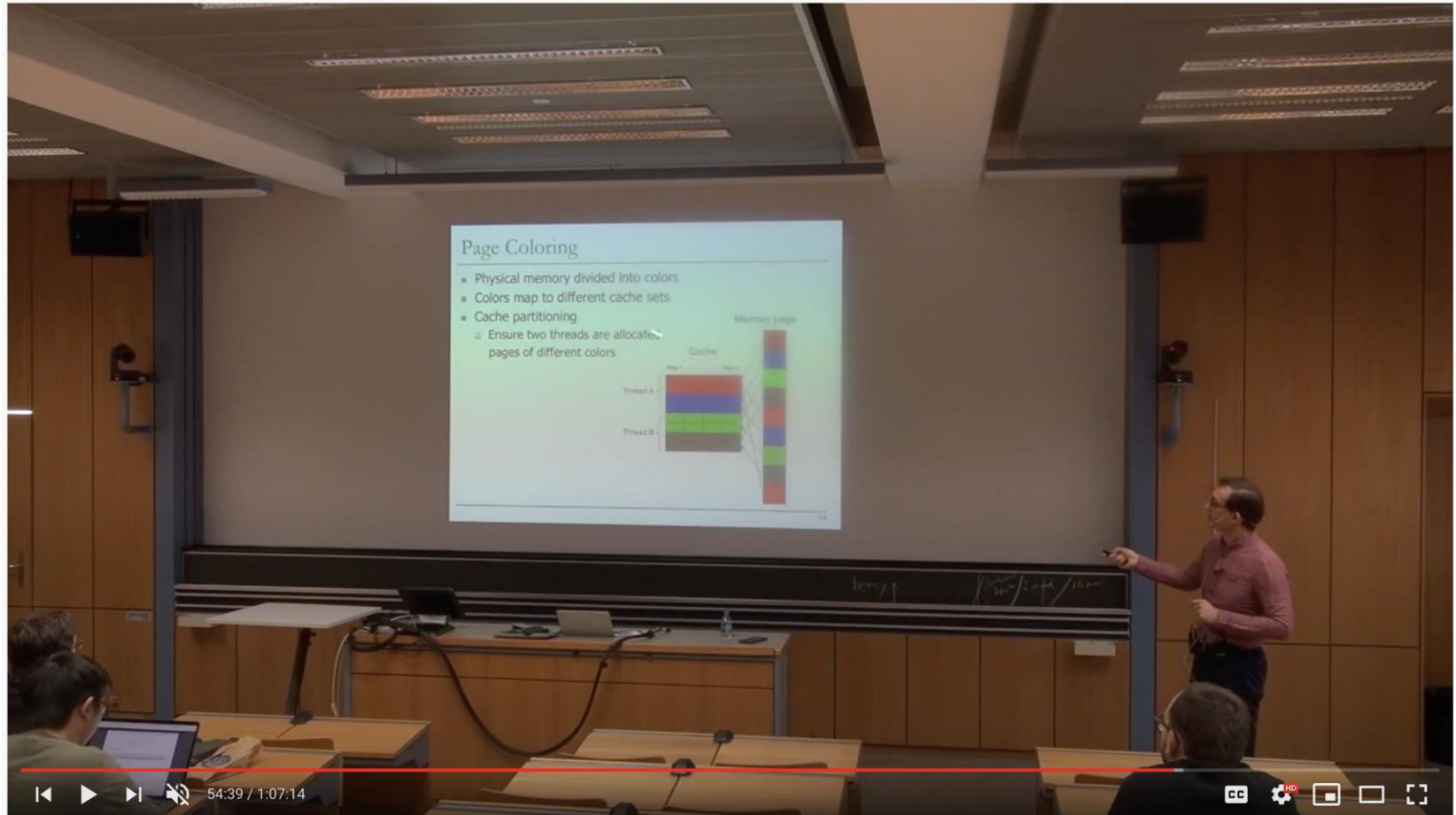


Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Multi-Core Cache Management



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 18b: Multi-Core Cache Management (ETH Zürich, Fall 2018)

744 views • Nov 23, 2018

12 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Multi-Core Cache Management

Approaches to Reuse Prediction

Use program counter or memory region information.

1. Group Blocks
2. Learn group behavior
3. Predict reuse

1. Same group $\nearrow$ same reuse behavior
2. No control over number of high-reuse blocks

ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 19a: Multi-Core Cache Management II (ETH Zürich, Fall 2018)

293 views • Dec 2, 2018

6 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

https://www.youtube.com/watch?v=Siz86_PD4w&list=PL5Q2soXY2Zi9JXe3ywQMhylk_d5dl-TM7&index=30

Lectures on Multi-Core Cache Management

- **Computer Architecture, Fall 2018, Lecture 18b**
 - Multi-Core Cache Management (ETH, Fall 2018)
 - https://www.youtube.com/watch?v=c9FhGRB3HoA&list=PL5Q2soXY2Zi9JXe3ywQMhylk_d5dI-TM7&index=29

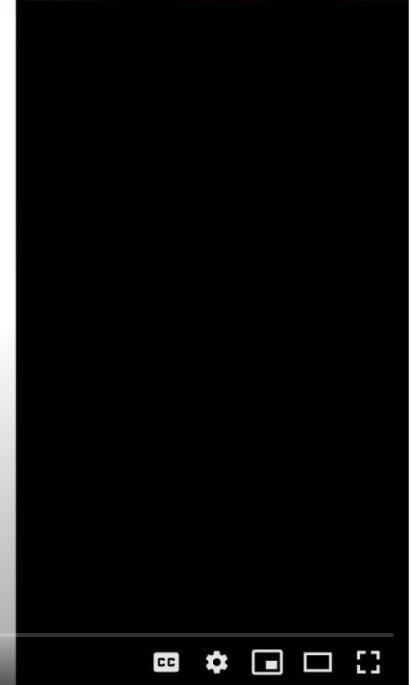
- **Computer Architecture, Fall 2018, Lecture 19a**
 - Multi-Core Cache Management II (ETH, Fall 2018)
 - https://www.youtube.com/watch?v=Siz86_PD4w&list=PL5Q2soXY2Zi9JXe3ywQMhylk_d5dI-TM7&index=30

- **Computer Architecture, Fall 2017, Lecture 15**
 - Multi-Core Cache Management (ETH, Fall 2017)
 - https://www.youtube.com/watch?v=7_Tqlw8gxOU&list=PL5Q2soXY2Zi9OhoVQBXYFIZywZXCPI4M_&index=17

Lectures on Memory Resource Management

QoS-Aware Memory Systems: Challenges

- How do we **reduce inter-thread interference**?
 - ❑ Improve system performance and core utilization
 - ❑ Reduce request serialization and core starvation
- How do we **control inter-thread interference**?
 - ❑ Provide mechanisms to enable system software to enforce QoS policies
 - ❑ While providing high system performance
- How do we **make the memory system configurable/flexible**?
 - ❑ Enable flexible mechanisms that can achieve many goals
 - Provide fairness or throughput when needed
 - Satisfy performance guarantees when needed



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 11b: Memory Interference and QoS (ETH Zürich, Fall 2020)

735 views • Oct 31, 2020

👍 14 🗨️ 0 ➦ SHARE ≡+ SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

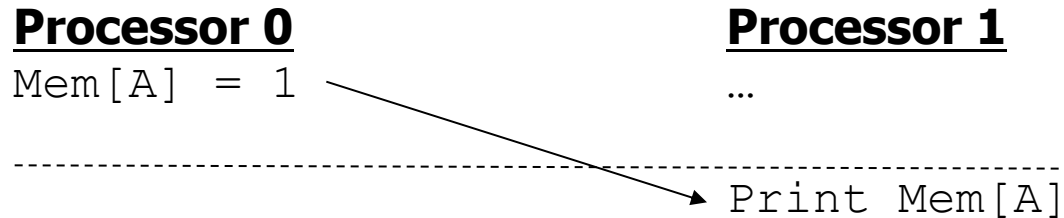
Lectures on Memory Resource Management

- **Computer Architecture, Fall 2020, Lecture 11a**
 - Memory Controllers (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=TeG773OgiMQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=20>
- **Computer Architecture, Fall 2020, Lecture 11b**
 - Memory Interference and QoS (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=0nnI807nCkc&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=21>
- **Computer Architecture, Fall 2020, Lecture 13**
 - Memory Interference and QoS II (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=Axye9VqQT7w&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=26>
- **Computer Architecture, Fall 2020, Lecture 2a**
 - Memory Performance Attacks (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=VJzZbwgBfy8&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=2>

Cache Coherence

Shared Memory Model

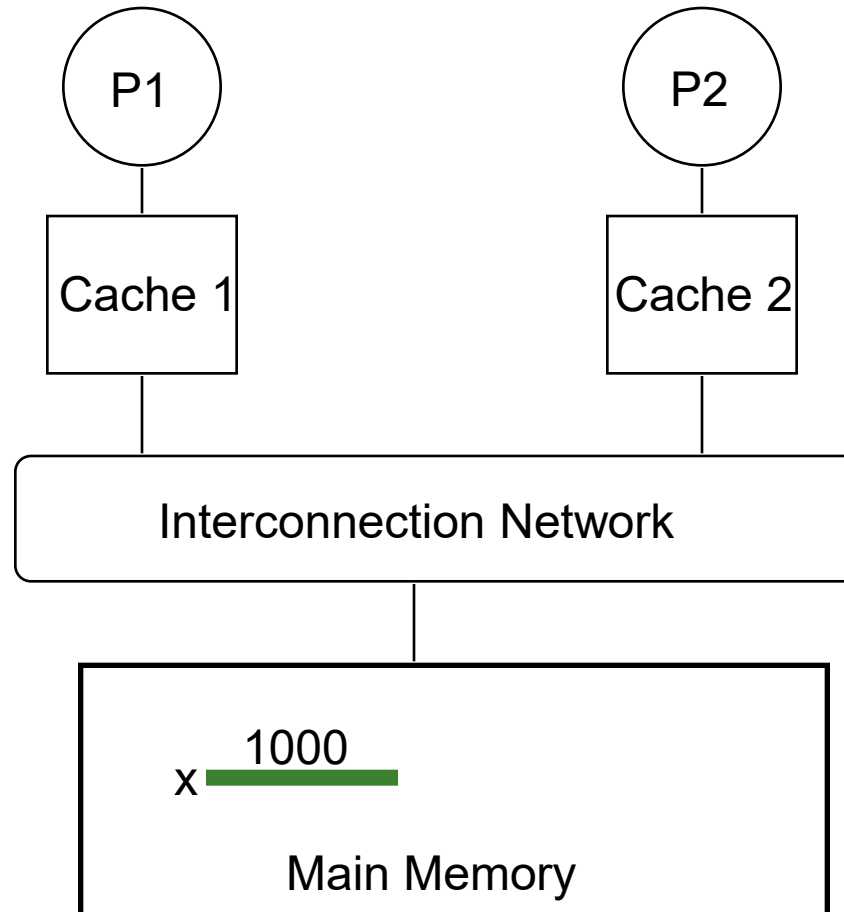
- Threads in parallel programs communicate through *shared memory*
- Thread 0 writes to an address, followed by Thread 1 reading
 - This implies communication between the two



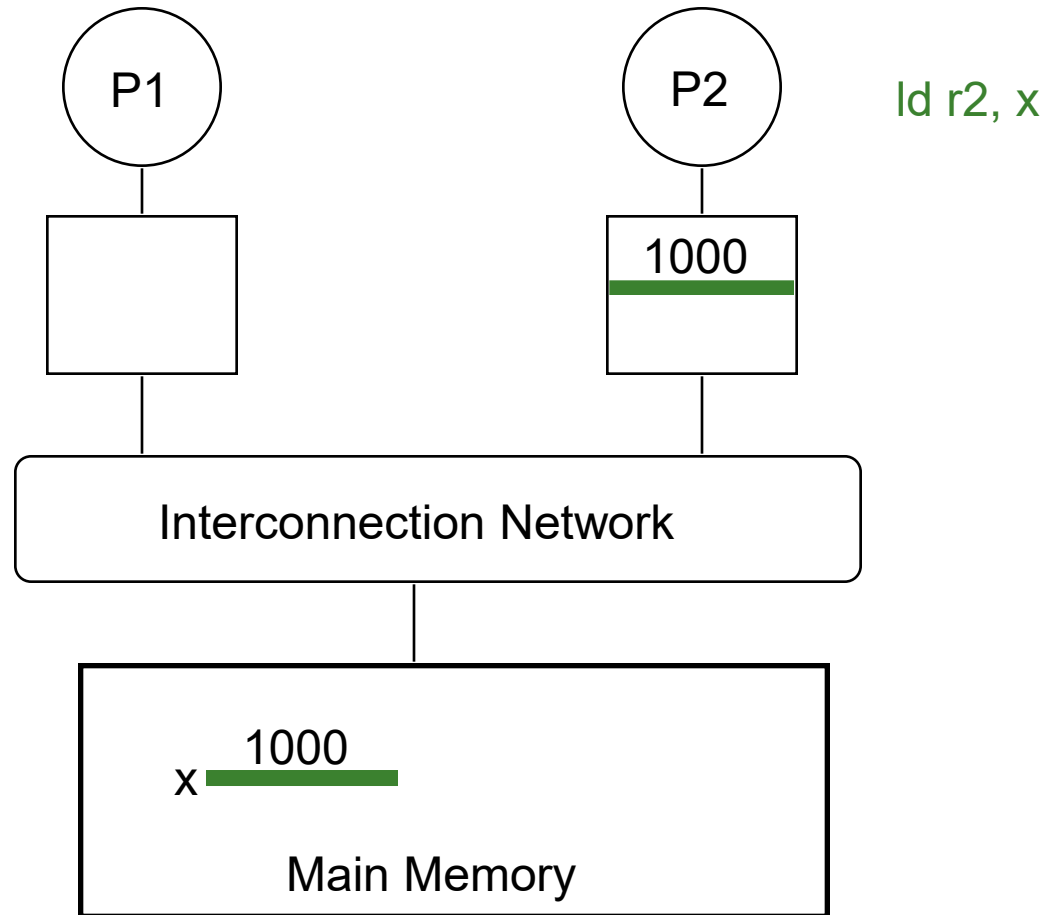
- Each read should receive the value last written by any processor
 - This requires synchronization between threads (i.e., what does last written mean?)
- What if Mem[A] is cached (at either processor)?

Cache Coherence

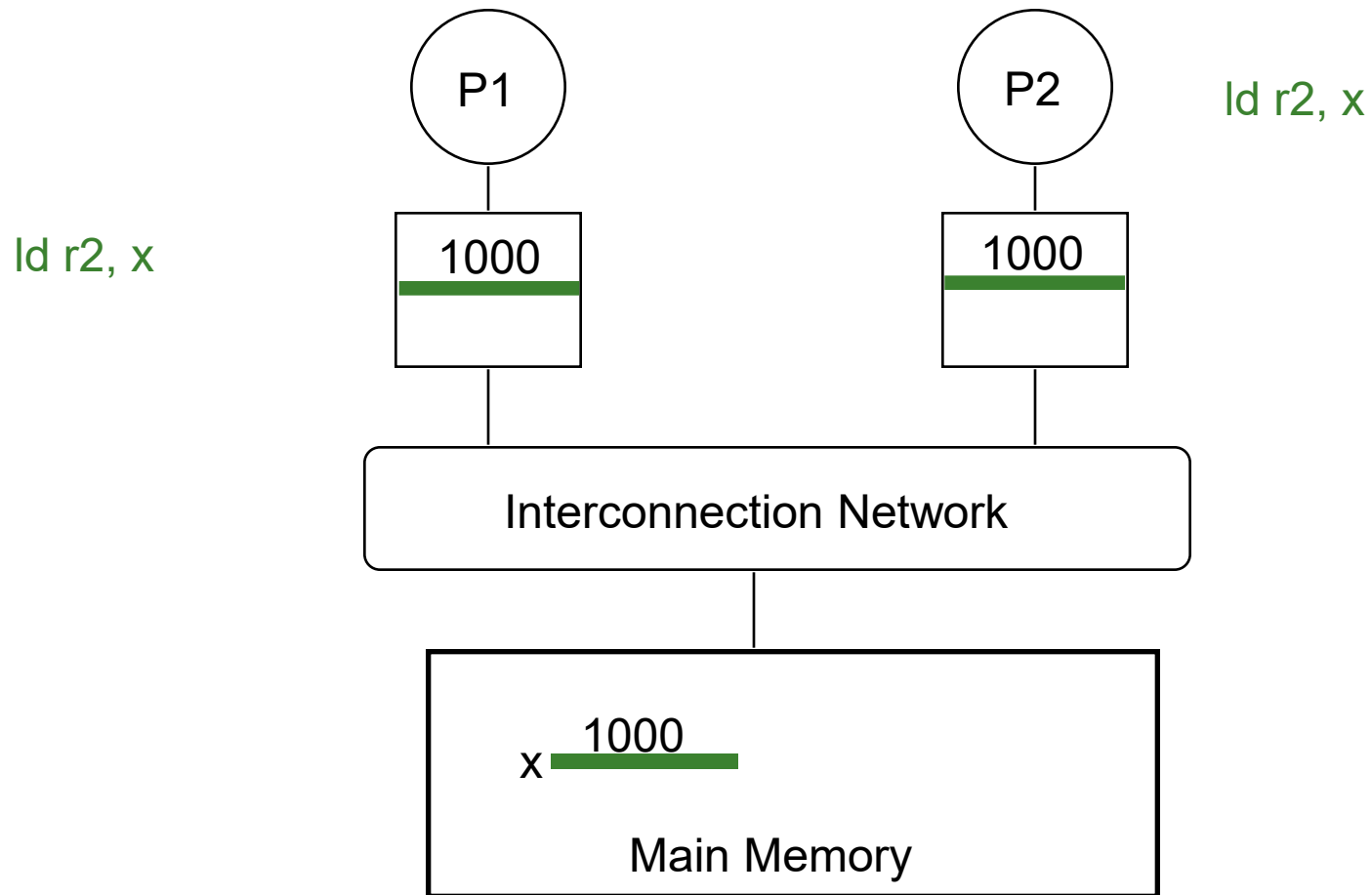
- Basic question: If multiple processors cache the same block, how do they ensure they all see a consistent state?



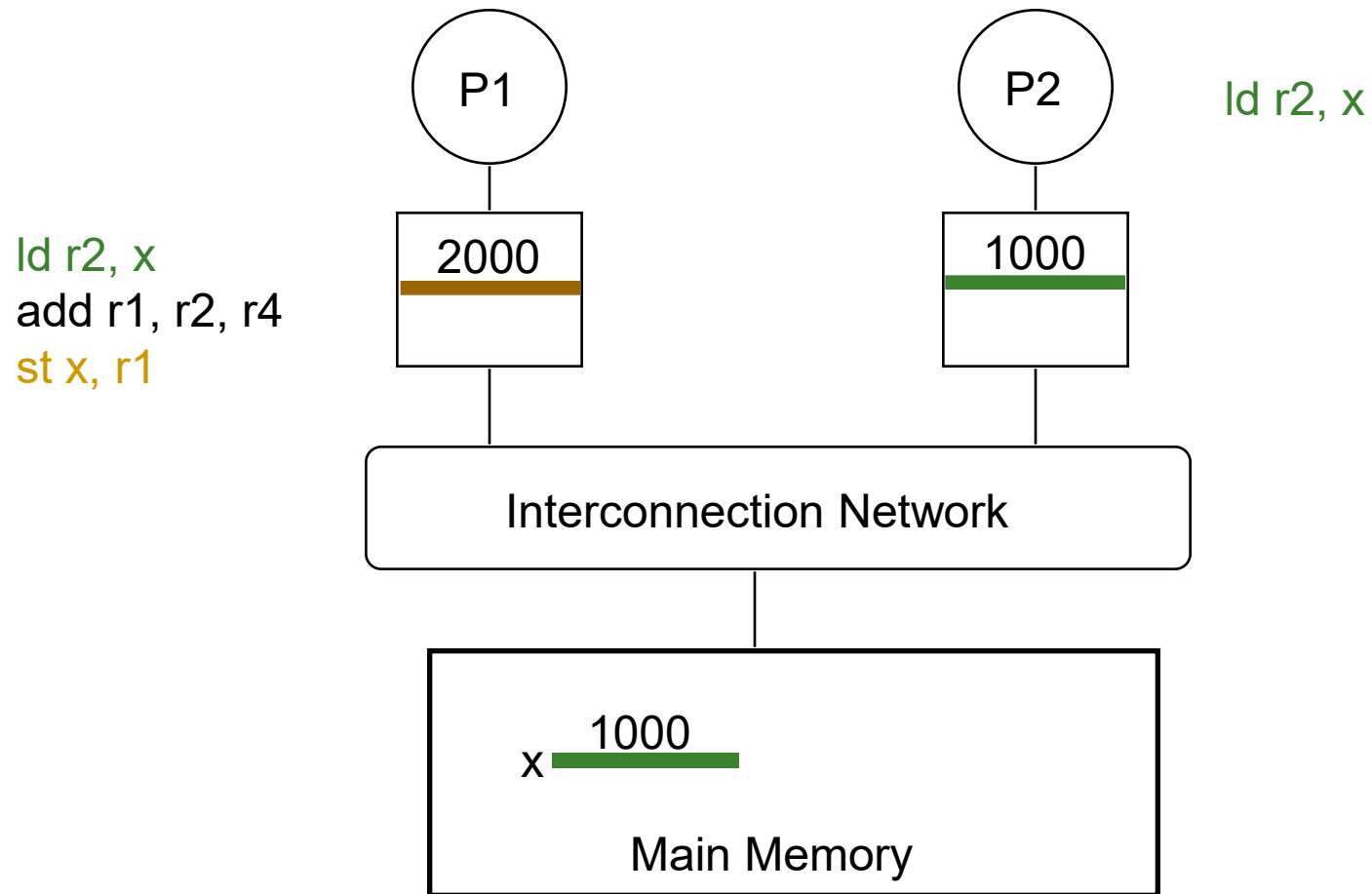
The Cache Coherence Problem



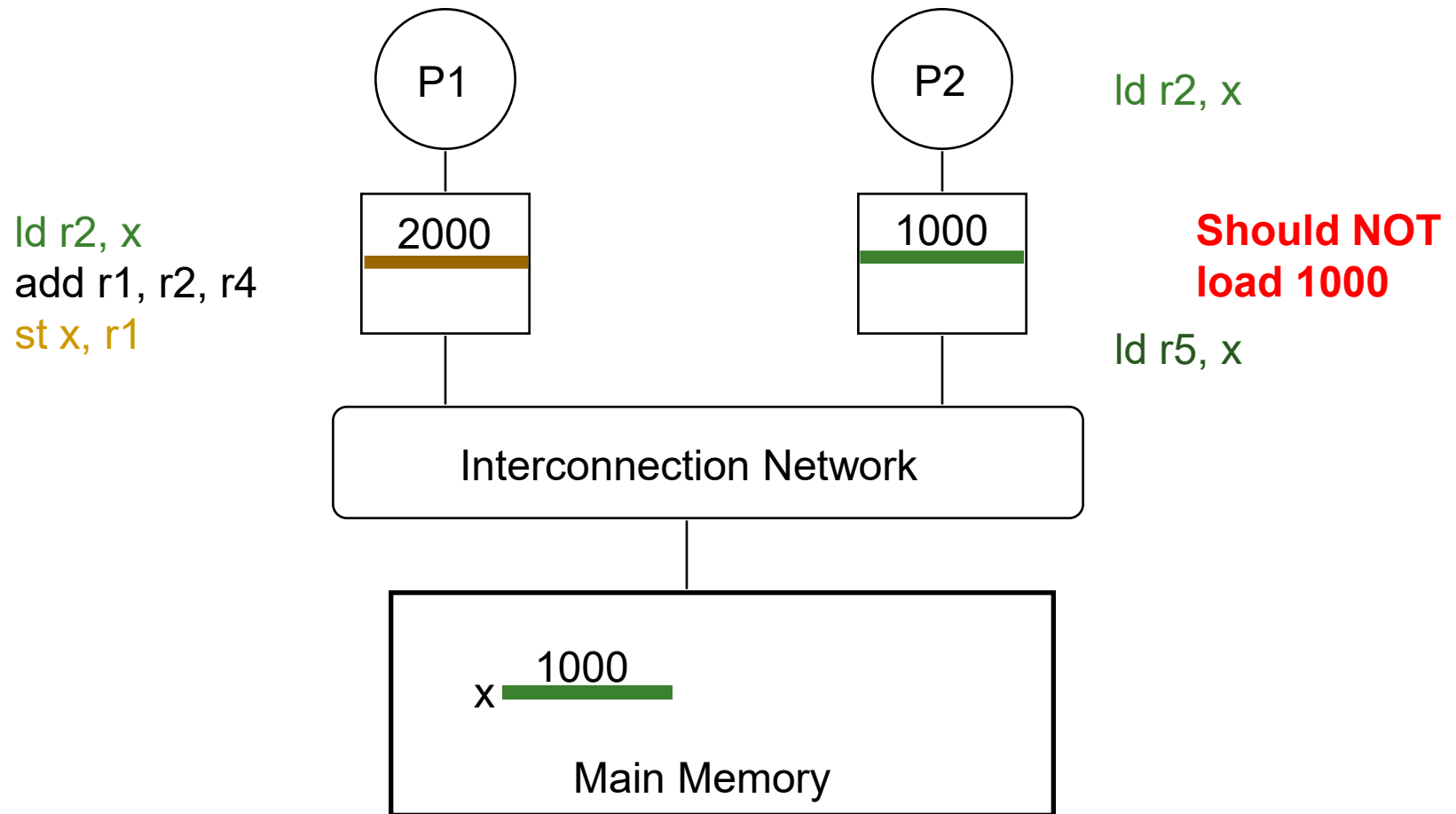
The Cache Coherence Problem



The Cache Coherence Problem

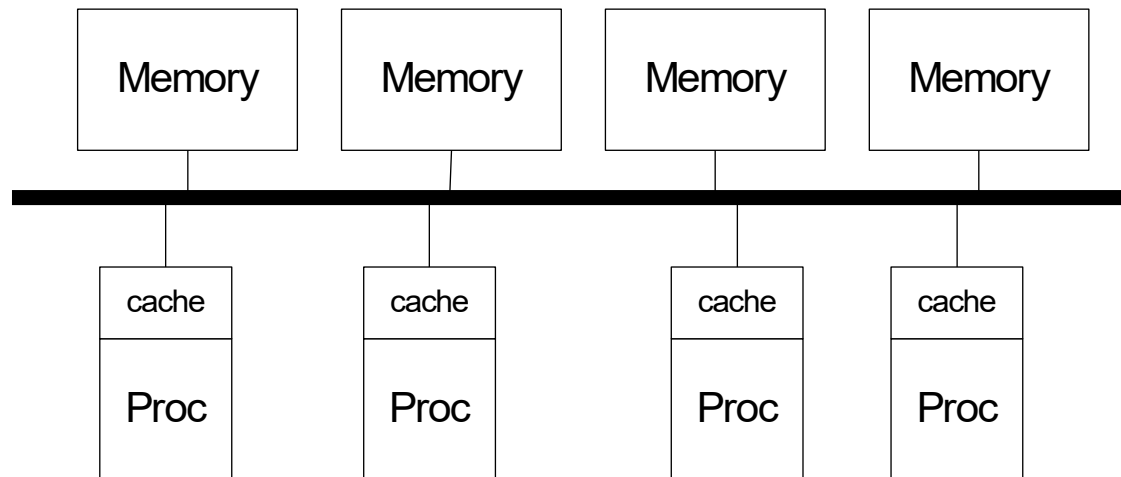


The Cache Coherence Problem



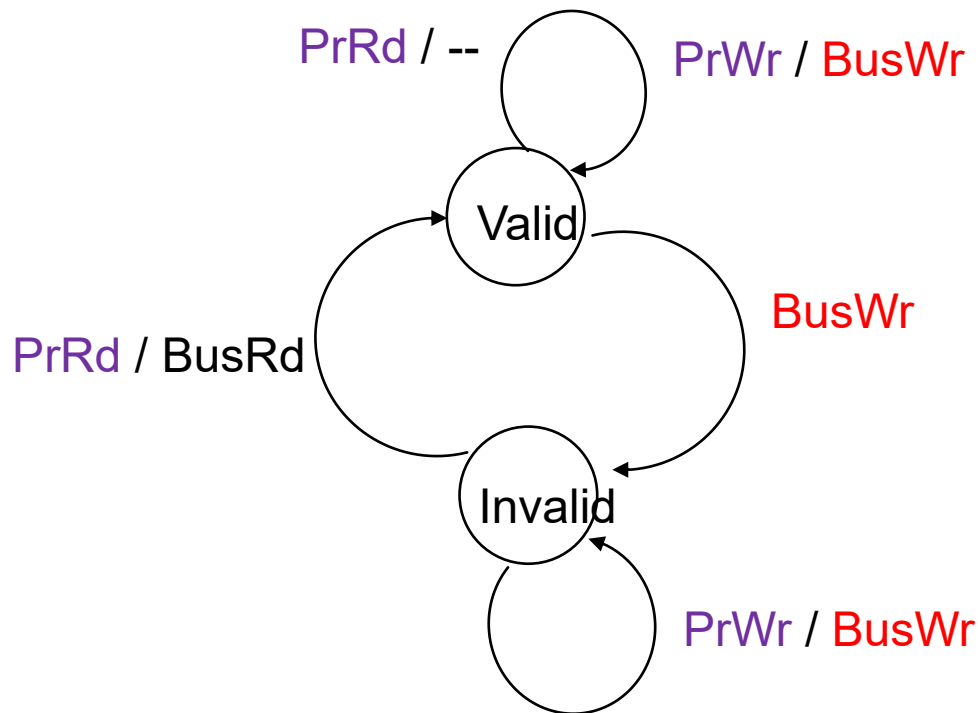
Broadcast-Based Hardware Cache Coherence

- Basic idea:
 - A processor/cache broadcasts its write/update to a cache block to all other processors
 - Another processor/cache that has the block either invalidates or updates its local copy of the block



A Very Simple Coherence Scheme (VI)

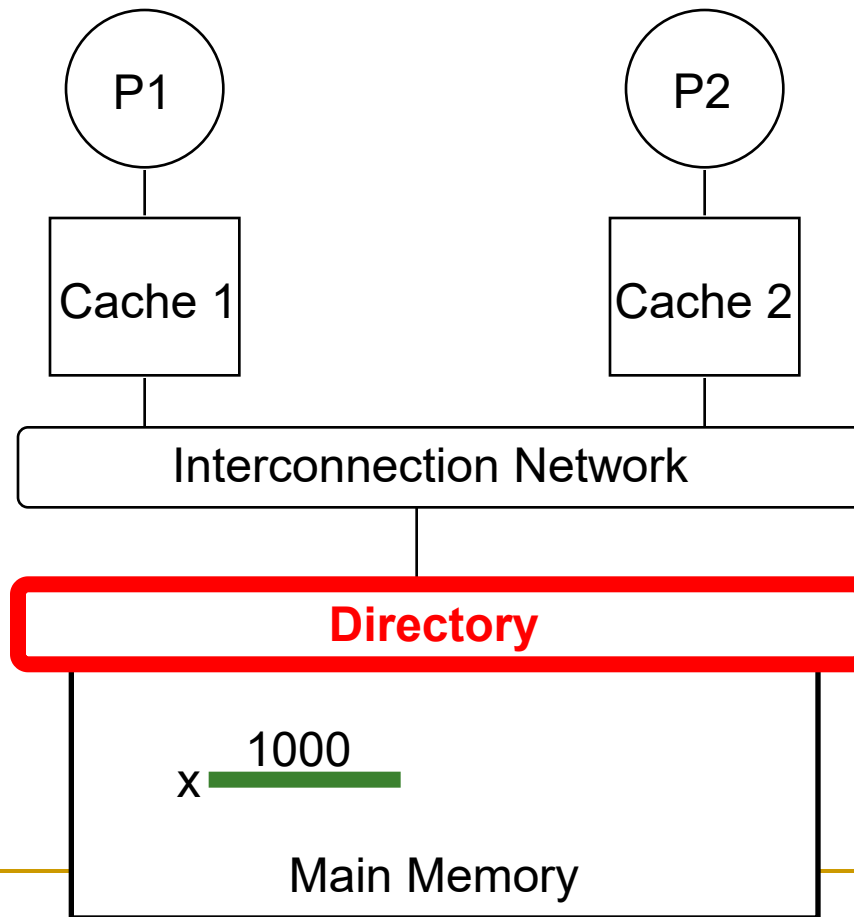
- Idea: All caches “snoop” (observe) each other’s write/read operations. If a processor writes to a block, all others invalidate the block.
- A simple protocol:



- Assumption: Write-through, no-write-allocate cache
- Actions of the local processor on the cache block: **PrRd, PrWr**
- Actions that are broadcast on the bus for the block: **BusRd, BusWr**

Directory-Based Cache Coherence

- Idea: A central directory keeps track of which caches contain every possible cache block. Every cache consults this directory to ensure data is kept coherent.

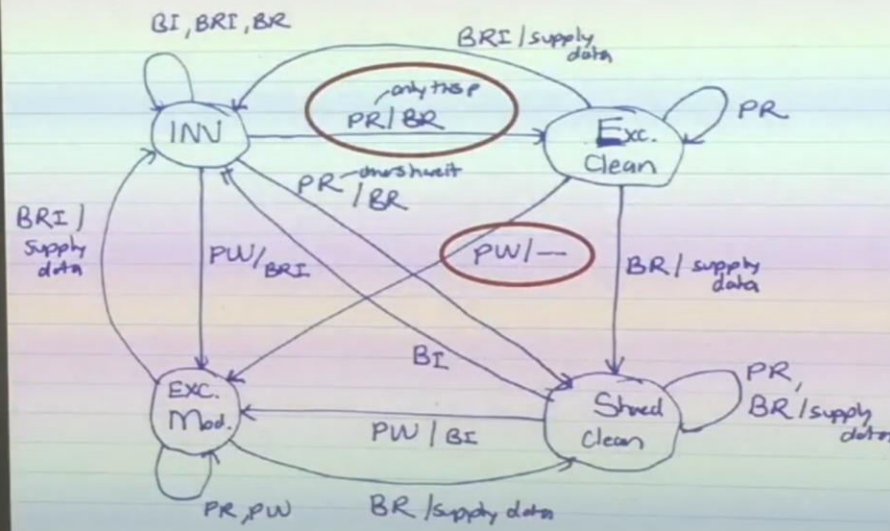


Directory-Based Cache Coherence

- Idea: A central directory keeps track of which caches contain every possible cache block. Every cache consults this directory to ensure data is kept coherent.
- An example mechanism:
 - For each cache block in memory, store $P+1$ bits in directory
 - One bit for each cache, indicating whether the block is in cache
 - Exclusive bit: indicates that a cache has the only copy of the block and can update it without notifying others
 - On a read: set the cache's bit in the directory entry for the block and arrange the supply of the block into the cache
 - On a write: invalidate the block in all caches that have the block and reset their bits in directory entry for the block
 - Have an "exclusive bit" associated with each block in each cache (so that the cache can update the exclusive block silently)

Lecture on Cache Coherence

MESI State Machine



Computer Architecture - Lecture 21: Cache Coherence (ETH Zürich, Fall 2020)

1,419 views • Dec 4, 2020

27 0 SHARE SAVE ...

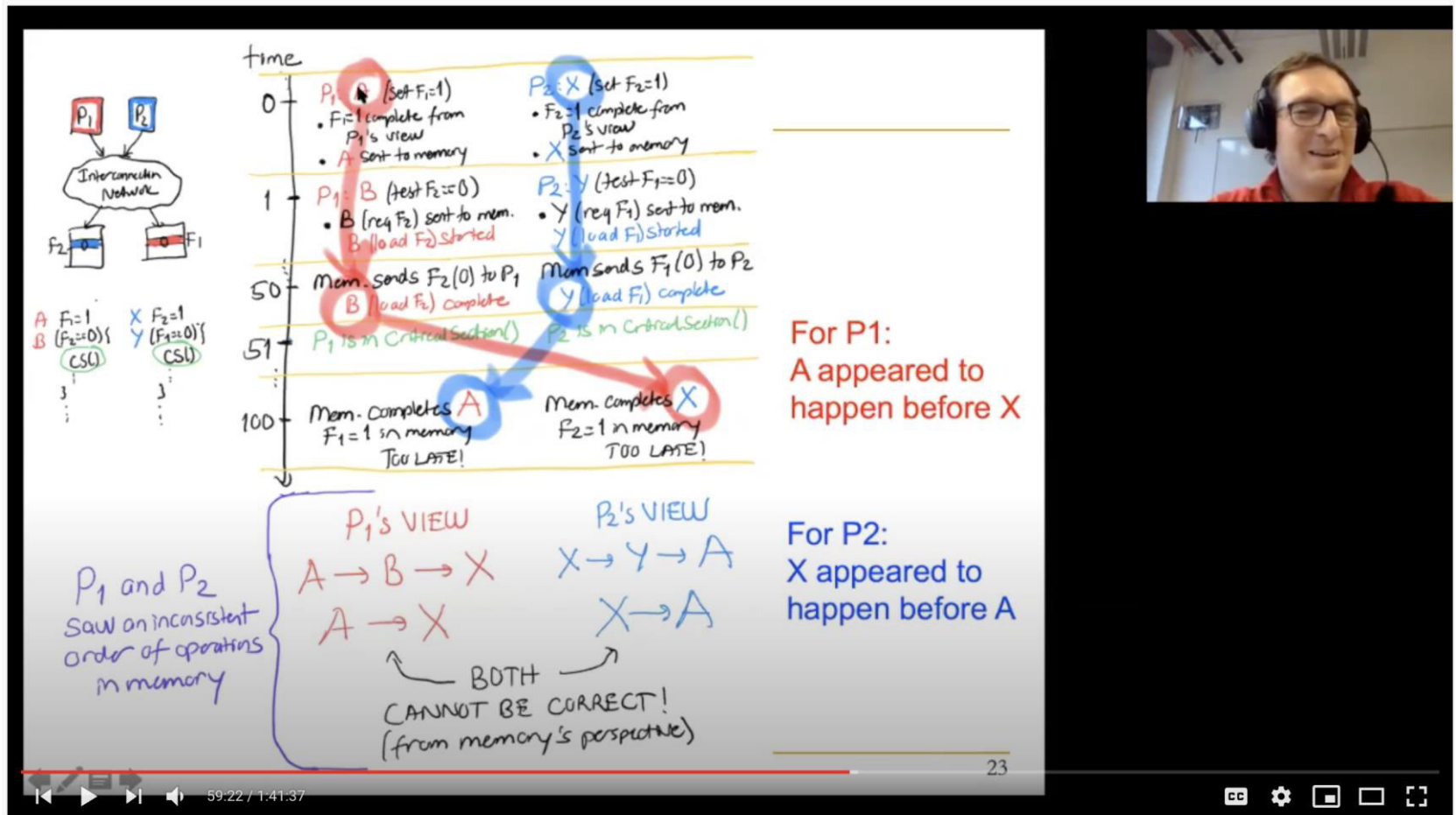


Onur Mutlu Lectures
16.3K subscribers

ANALYTICS

EDIT VIDEO

Lecture on Memory Ordering & Consistency



Computer Architecture - Lecture 20: Memory Ordering (Memory Consistency) (ETH Zürich, Fall 2020)

976 views • Dec 4, 2020

22 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lecture on Cache Coherence & Consistency

■ Computer Architecture, Fall 2020, Lecture 21

- ❑ Cache Coherence (ETH, Fall 2020)
- ❑ <https://www.youtube.com/watch?v=T9WlyezeaII&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=38>

■ Computer Architecture, Fall 2020, Lecture 20

- ❑ Memory Ordering & Consistency (ETH, Fall 2020)
- ❑ <https://www.youtube.com/watch?v=Suy09mzTbiQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=37>

■ Computer Architecture, Spring 2015, Lecture 28

- ❑ Memory Consistency & Cache Coherence (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=JfjT1a0vi4E&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=32>

■ Computer Architecture, Spring 2015, Lecture 29

- ❑ Cache Coherence (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=X6DZchnMYcw&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=33>

Digital Design & Computer Arch.

Lecture 24a: Advanced Caches

Prof. Onur Mutlu

ETH Zürich

Spring 2026

22 May 2026

Additional Slides: Cache Coherence

Two Cache Coherence Methods

- ❑ How do we ensure that the proper caches are updated?
- ❑ **Snoopy Bus** [Goodman ISCA 1983, Papamarcos+ ISCA 1984]
 - Bus-based, *single point of serialization for all memory requests*
 - Processors observe other processors' actions
 - ❑ E.g.: P1 makes “read-exclusive” request for A on bus, P0 sees this and invalidates its own copy of A
- ❑ **Directory** [Censier and Feautrier, IEEE ToC 1978]
 - *Single point of serialization per block*, distributed among nodes
 - Processors make explicit requests for blocks
 - Directory tracks which caches have each block
 - Directory coordinates invalidations and updates
 - ❑ E.g.: P1 asks directory for exclusive copy, directory asks P0 to invalidate, waits for ACK, then responds to P1

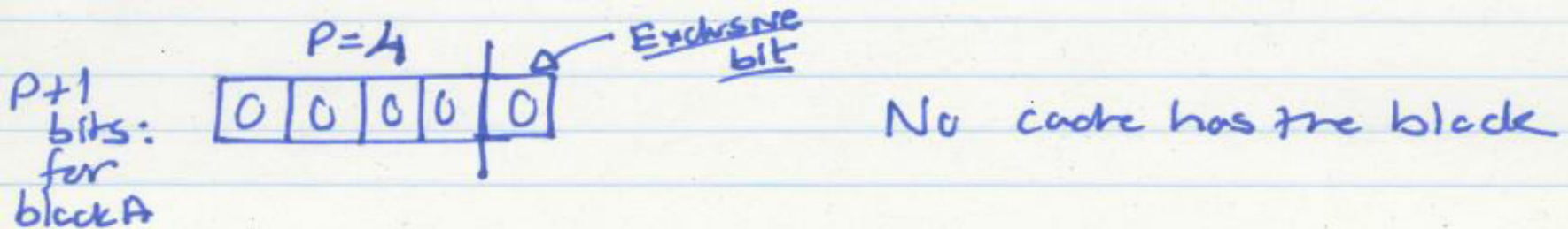
Directory Based Coherence

- Idea: A logically-central directory keeps track of where the copies of each cache block reside. Caches consult this directory to ensure coherence.

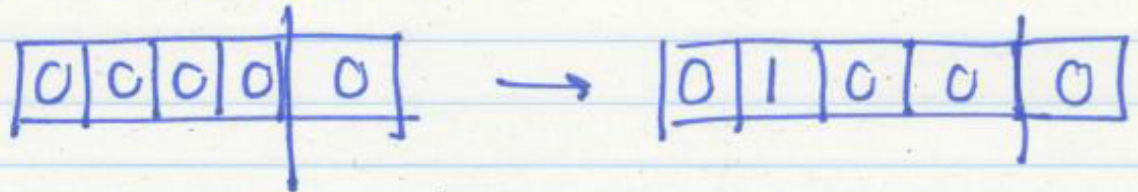
- An example mechanism:
 - For each cache block in memory, store $P+1$ bits in directory
 - One bit for each cache, indicating whether the block is in cache
 - Exclusive bit: indicates that a cache has the only copy of the block and can update it without notifying others
 - On a read: set the cache's bit and arrange the supply of data
 - On a write: invalidate all caches that have the block and reset their bits
 - Have an "exclusive bit" associated with each block in each cache (so that the cache can update the exclusive block silently)

Directory Based Coherence Example (I)

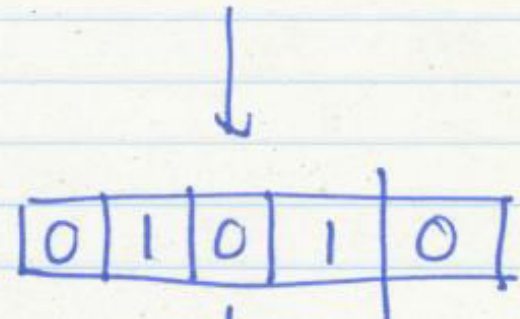
Example directory based scheme



① P_1 takes a read miss to block A

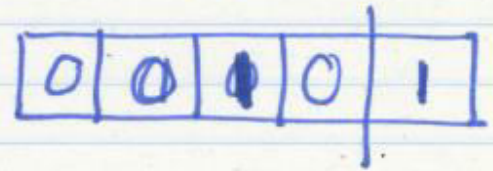


② P_3 takes a read miss



③ P₂ takes a write miss

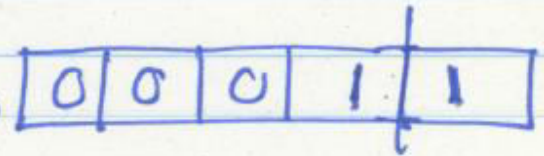
- invalidate P₁ & P₃'s caches
- write request → P₂ has the exclusive copy of the block now. Set the Exclusive bit



- P₂ can now update the block without notifying any other processor or the directory
- P₂ needs to have a bit in its cache indicating it can perform exclusive updates to that block
 - private/exclusive bit per cache block

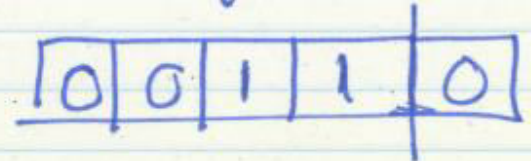
④ P₃ takes a write miss

- Mem ~~Controller~~ requests ~~the~~ block from P₂
- Mem Controller gives block to P₃
- P₂ invalidates its copy



⑤ P₂ takes a read miss

- P₃ supplies it



Maintaining Coherence

- Need to guarantee that all processors see a consistent value (i.e., consistent updates) for the same memory location
- Writes to location A by P0 should be seen by P1 (eventually), and all writes to A should appear in some order
- Coherence needs to provide:
 - **Write propagation:** guarantee that updates will propagate
 - **Write serialization:** provide a consistent order seen by all processors for the same memory location
- Need a global point of serialization for this write ordering

Coherence: Update vs. Invalidate

- How can we *safely update replicated data*?
 - Option 1 (Update protocol): push an update to all copies
 - Option 2 (Invalidate protocol): ensure there is only one copy (local), update it
- **On a Read:**
 - If local copy is Invalid, put out request
 - (If another node has a copy, it returns it, otherwise memory does)

Coherence: Update vs. Invalidate (II)

■ On a Write:

- ❑ Read block into cache as before

Update Protocol:

- ❑ Write to block, and simultaneously broadcast written data and address to sharers
- ❑ (Other nodes update the data in their caches if block is present)

Invalidate Protocol:

- ❑ Write to block, and simultaneously broadcast invalidation of address to sharers
- ❑ (Other nodes invalidate block in their caches if block is present)

Update vs. Invalidate Tradeoffs

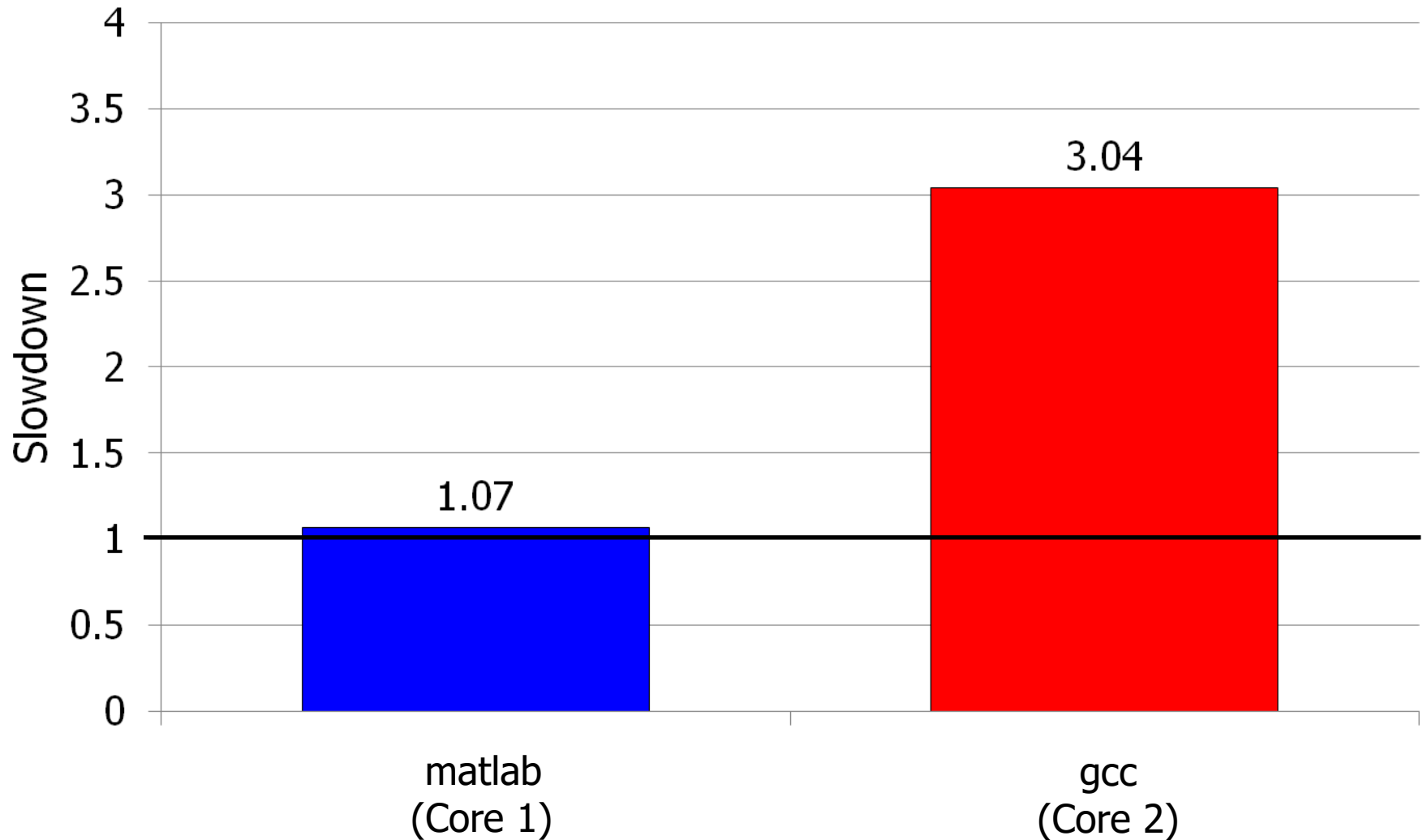
- Which one is better? Update or invalidate?
 - Write frequency and sharing behavior are critical
- **Update**
 - + If sharer set is constant and updates are infrequent, avoids the cost of invalidate-reacquire (broadcast update pattern)
 - If data is rewritten without intervening reads by other cores, updates would be useless
 - Write-through cache policy → bus can become a bottleneck
- **Invalidate**
 - + After invalidation, core has exclusive access rights
 - + Only cores that keep reading after each write retain a copy
 - If write contention is high, leads to ping-ponging (rapid invalidation-reacquire traffic from different processors)

Additional Slides: Memory Interference

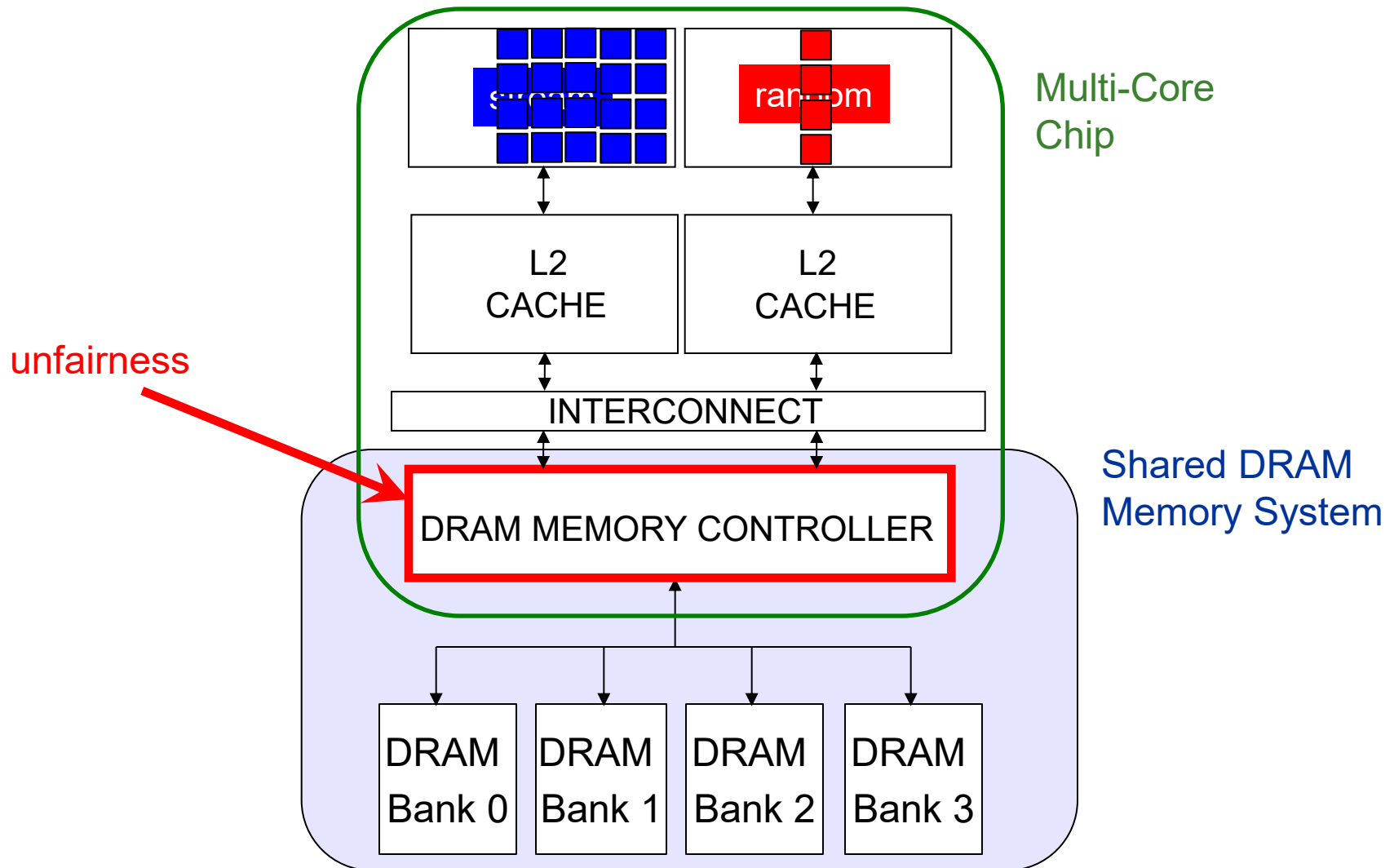
Inter-Thread/Application Interference

- Problem: Threads share the memory system, but memory system does not distinguish between threads' requests
- Existing memory systems
 - ❑ Free-for-all, shared based on demand
 - ❑ Control algorithms thread-unaware and thread-unfair
 - ❑ Aggressive threads can deny service to others
 - ❑ Do not try to reduce or control inter-thread interference

Unfair Slowdowns due to Interference



Uncontrolled Interference: An Example



A Memory Performance Hog

```
// initialize large arrays A, B  
for (j=0; j<N; j++) {  
    index = j*linesize; streaming  
    A[index] = B[index];  
    ...  
}
```

STREAM

- Sequential memory access
- Very high row buffer locality (96% hit rate)
- Memory intensive

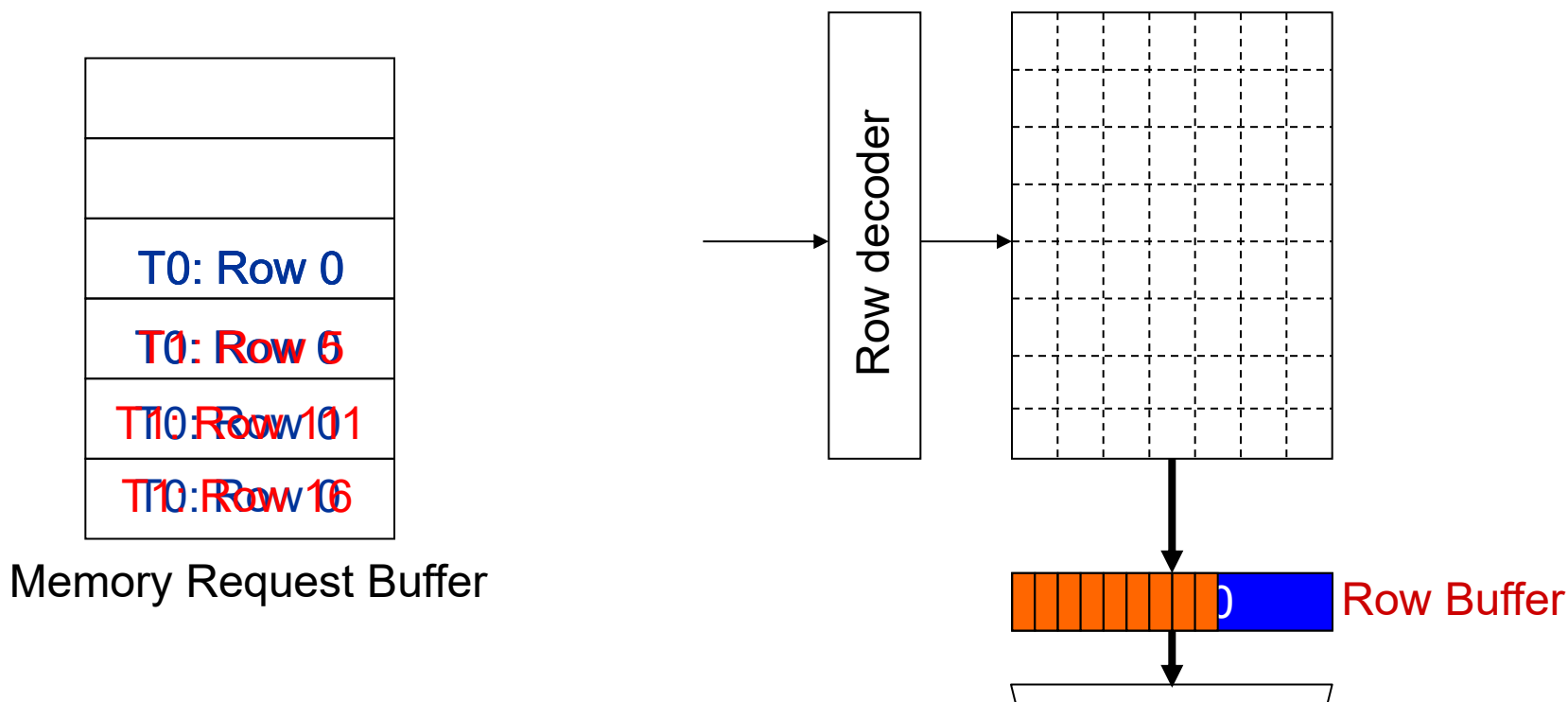
```
// initialize large arrays A, B  
for (j=0; j<N; j++) {  
    index = rand(); random  
    A[index] = B[index];  
    ...  
}
```

RANDOM

- Random memory access
- Very low row buffer locality (3% hit rate)
- Similarly memory intensive

Moscibroda and Mutlu, “[Memory Performance Attacks](#),” USENIX Security 2007.

What Does the Memory Hog Do?



Row size: 8KB, cache block size: 64B
128 (8KB/64B) requests of T0 serviced before T1

Moscibroda and Mutlu, “[Memory Performance Attacks](#),” USENIX Security 2007.

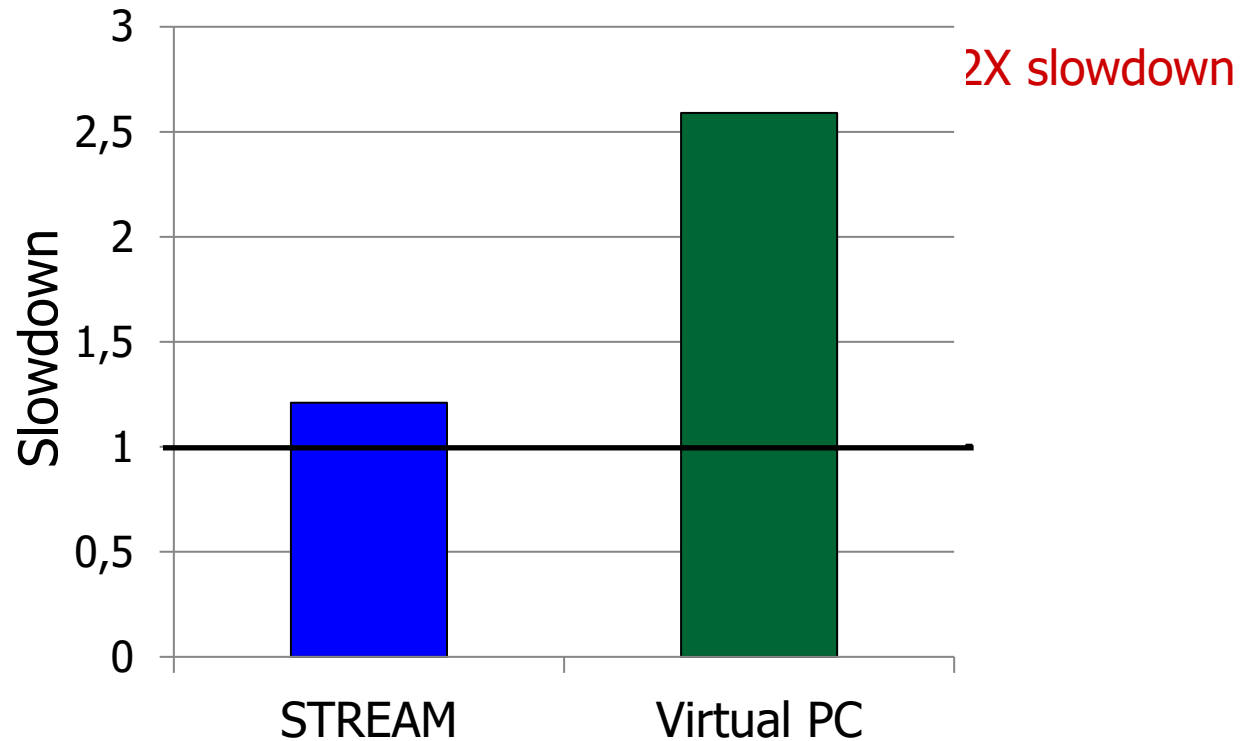
DRAM Controllers

- A row-conflict memory access takes significantly longer than a row-hit access
- Current controllers take advantage of the row buffer
- Commonly used scheduling policy (FR-FCFS) [Rixner 2000]*
 - (1) Row-hit first: Service row-hit memory accesses first
 - (2) Oldest-first: Then service older accesses first
- This scheduling policy aims to maximize DRAM throughput
 - But, it is unfair when multiple threads share the DRAM system

*Rixner et al., “Memory Access Scheduling,” ISCA 2000.

*Zuravleff and Robinson, “Controller for a synchronous DRAM ...,” US Patent 5,630,096, May 1997.

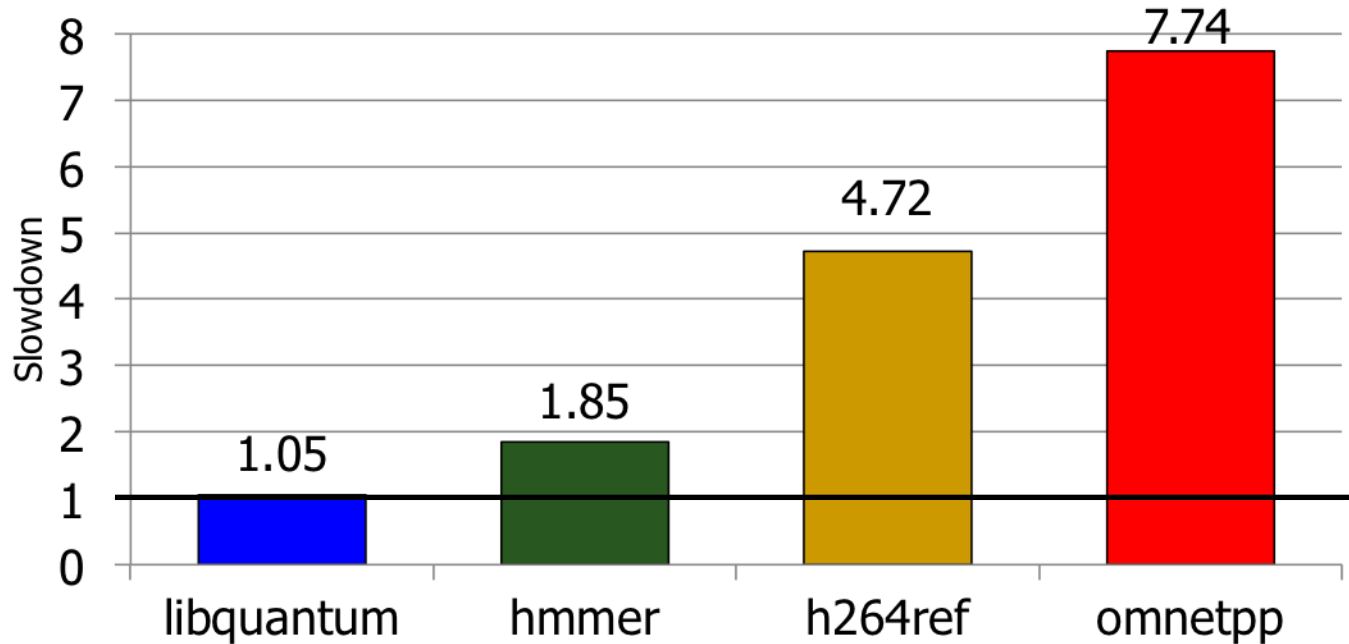
Effect of the Memory Performance Hog



Results on Intel Pentium D running Windows XP
(Similar results for Intel Core Duo and AMD Turion, and on Fedora Linux)

Moscibroda and Mutlu, “[Memory Performance Attacks](#),” USENIX Security 2007.

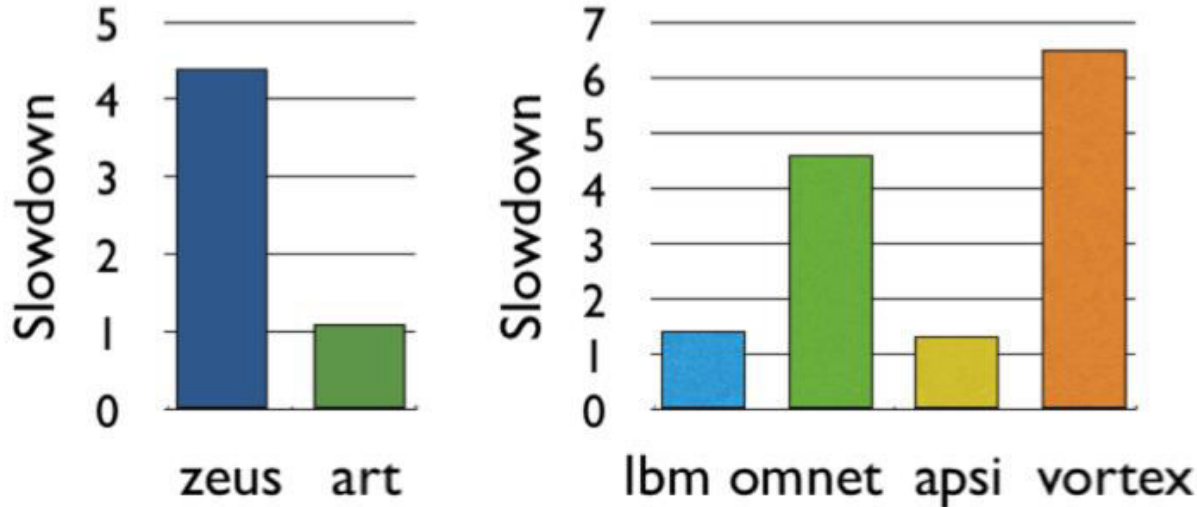
Greater Problem with More Cores



- Vulnerable to denial of service (DoS)
- Unable to enforce priorities or SLAs
- Low system performance

Uncontrollable, unpredictable system

Greater Problem with More Cores



- Vulnerable to denial of service (DoS)
- Unable to enforce priorities or SLAs
- Low system performance

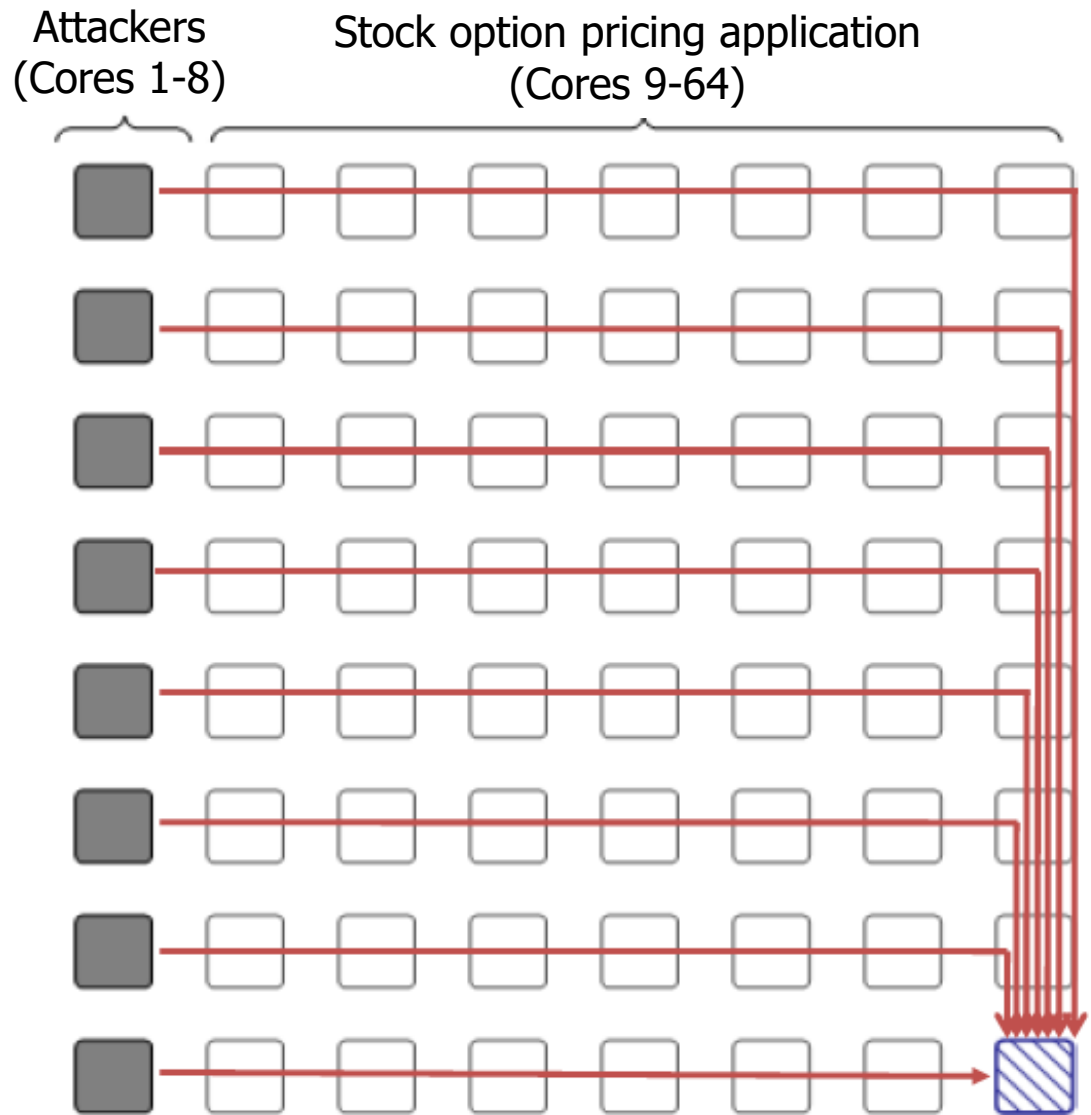
Uncontrollable, unpredictable system

Distributed DoS in Networked Multi-Core Systems

Cores connected via
packet-switched
routers on chip

~5000X latency increase

Grot, Hestness, Keckler, Mutlu,
"Preemptive virtual clock: A Flexible,
Efficient, and Cost-effective QOS
Scheme for Networks-on-Chip,"
MICRO 2009.



More on Memory Performance Attacks

- Thomas Moscibroda and Onur Mutlu,
"Memory Performance Attacks: Denial of Memory Service in Multi-Core Systems"
*Proceedings of the 16th USENIX Security Symposium (**USENIX SECURITY**), pages 257-274, Boston, MA, August 2007. Slides (ppt)*

Memory Performance Attacks: Denial of Memory Service in Multi-Core Systems

Thomas Moscibroda Onur Mutlu
Microsoft Research
{moscitho,onur}@microsoft.com

More on Interconnect Based Starvation

- Boris Grot, Stephen W. Keckler, and Onur Mutlu,
"Preemptive Virtual Clock: A Flexible, Efficient, and Cost-effective QOS Scheme for Networks-on-Chip"
*Proceedings of the 42nd International Symposium on Microarchitecture (**MICRO**), pages 268-279, New York, NY, December 2009. Slides (pdf)*

Preemptive Virtual Clock: A Flexible, Efficient, and Cost-effective QOS Scheme for Networks-on-Chip

Boris Grot

Stephen W. Keckler

Onur Mutlu[†]

Department of Computer Sciences
The University of Texas at Austin
{bgrot, skeckler}@cs.utexas.edu

[†]Computer Architecture Laboratory (CALCM)
Carnegie Mellon University
onur@cmu.edu